分类号：TP242　　单位代码：10335

密　级：无　　学　　号：21910101

浙江大学

硕士学位论文

中文论文题目：时序逻辑约束下的多机器人任务分配与协同优化

英文论文题目：**Multi-robot Task Allocation and Coordination under Temporal Logic Constraints**

申请人姓名：白若飞

指导教师：郑荣濠副教授

学科 (专业)：控制理论与控制工程

研究方向：多机器人协同

所在学院：电气工程学院

论文递交日期　**2022 年 01 月**

中国知网　https://www.cnki.net

时序逻辑约束下的
多机器人任务分配与协同优化

论文作者签名：白若飞

指导教师签名：邓辉煌

论文评阅人 1：隐名评阅人 1

评阅人 2：隐名评阅人 2

评阅人 3：隐名评阅人 3

评阅人 4：

评阅人 5：

答辩委员会主席：杨仕友/教授/浙江大学

委员 1：汪舒生/教授级高工/中建材智能自动化研究院

委员 2：张森林/教授/浙江大学

委员 3：

委员 4：

答辩日期 2022 年 3 月 8 日

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

致谢

硕士生涯即将结束，在杭州这个美丽的城市也已经生活了快七年。回顾过往的意气风发或是苦闷焦虑，当初稚嫩无虑的面庞现在变得成熟，心里留下的是对母校浙江大学的感激，对这一路走来遇到的老师和同学的感谢与祝福。

首先感谢我的导师郑荣濠老师。从我进课题组开始，受到郑老师的指导已快 4 年。郑老师崇高的学术追求，严谨的科研态度，浓厚的科研热情，影响了我做科研的态度和对学术的追求。在过去的科研中，郑老师一步步指导我如何从现有的工作中寻找有价值的研究点，如何将预期目标与日常工作相结合，如何保持科研的效率等，使我从思维和能力上都有了很大的提升。感谢刘妹琴老师对我的研究工作直中要害的点评和坦率真诚的建议，让我能及时意识到自己存在的问题。感谢张森林老师对我的鼓励与教诲，让我懂得很多做人做事的道理。感谢何衍老师对我科研的关注与建议，何老师严谨细致的授课风格对我有很大影响。感谢董山玲老师对我在学术深造上的建议以及日常的谈心交流。感谢樊臻老师对我的课题提出的宝贵意见。

感谢已经毕业的李存龙、孙博、刘凯鉴等师兄在我刚进入课题组时对我热情的帮助和对我长期的鼓励。感谢刚毕业的穆亚楠、楼冠廷、李星辉、赵恒飞等师兄师姐对我在科研学习生活上的指导与帮助。感谢施琳琳师姐对我的帮助，以及李琰、张佳欣、田一玮等师兄师姐们。感谢同届的博士生徐阳对我在撰写论文和科研思路等方面的帮助，以及进入课题组以来热心组织的集体活动。感谢王根檑、陈喜来、卢晨、胡双、葛志远等同学一起的日常讨论和学习。感谢霍嘉熹、叶卓勋、丁天逸、何翌、汤淼奕、孙祎芸、张航等师弟师妹给 218 带来的活力与快乐。祝福各位师兄师姐、师弟师妹，同窗好友，愿你们前程似锦，拥有美好的未来！

感谢舍友陈云龙、王继承、邓福强在这近三年来的共同生活与谈心交流。

最后，感谢女朋友和远在陕西的家人对我的学习与成长过程的关心与支持，前路更漫长，一起再出发。

白若飞

2022 年初于求是园

中国知网 https://www.cnki.net

摘要

随着社会生产生活智能化水平的提高，多机器人之间的协作广泛存在于智慧物流、无人工厂以及协同环境监测等领域，这对多机器人系统提出了复杂的任务要求，例如，工作环境中的任务需要多种异构机器人的协同配合才能满足，不同任务的执行需要满足一定的时序约束等。考虑时序约束的经典任务规划算法存在可拓展性较差及求解效率低等问题，适用场景受到很大限制。基于上述背景，本文提出了一种多机器人时序任务规划算法框架，用于求解时序逻辑约束下的多机器人任务协同规划问题，主要包括：

针对时序与协同约束下的机器人任务建模难和规划复杂度高等问题，本文利用线性时序逻辑语言来描述协同任务要求，基于相应的自动机理论识别该协同任务要求中可以相互独立执行的部分并进行独立性分解，从而降低实际执行中机器人之间的耦合程度；为进一步减小任务规划问题的规模，本文建立 SMT（Satisfiability Modulo Theories）模型来求解时序逻辑约束下的协同任务分配问题，并考虑通信受限的情况下如何满足协同任务之间的时序约束。此外，该方法利用 SMT 模型的迭代求解机制实现了对任务分配方案的快速遍历与筛除。

针对机器人同时具有个体任务要求和协同任务要求的复杂规划问题，本文提出一种可由个体机器人本地计算的执行策略求解算法，求解得到的执行策略可以满足任务的时序与协同要求，同时该本地求解过程也降低了规划算法的计算复杂度。本文进一步针对多机器人执行协同任务时产生的协同等待时间，提出一种基于分布式通信的执行策略迭代调整机制。机器人根据与其它机器人的信息交互，在本地迭代地调整自身的执行策略以优化在执行协同任务时的等待时间，进而有效降低多机器人的总时间损耗。

针对本文提出的多机器人时序任务规划框架，本文进行了相关的理论分析与仿真验证。本文在一定的假设下证明了该算法的完备性，并分析了其时间复杂度以及影响机器人执行策略最优性的因素。另外，本文在不同机器人数量、任务类型和环境规模下进行了大量的仿真实验，通过与基于全局乘积自动机的常见算法对比，验证了本文提出的多机器人时序任务规划算法在可拓展性和求解效率上的性能优势。实验结果也验证了本文提出的机器人执行策略迭代调整机制的快速收敛性与良好的策略优化效果。

关键词：多机器人；任务规划；线性时序逻辑；分布式通信；任务分解

中国知网 https://www.cnki.net

Abstract

Multi-robot cooperation widely exists in smart logistics, unmanned factories, and collaborative environment monitoring, which put forward complex task requirements for multi-robot systems. For example, tasks in the environment may require the cooperation of several heterogeneous robots, and the task execution may have to follow some temporal constraints. Classical task planning algorithms considering temporal constraints have poor scalability and low time efficiency, which limit their application scenarios. This thesis work proposes a multi-robot temporal task planning framework to mitigate the above problems. The main contributions are as follows:

To deal with the difficulty in modeling complex tasks and the high computational complexity of existing methods, we utilize the linear temporal logic language to describe robots' collaborative task requirements and identify the independent parts of the task requirements to reduce the coupling between robots in the execution. To reduce the size of planning problems, we develop an SMT (Satisfiability Modulo Theories) model to assign collaborative tasks to robots under temporal logic constraints, while considering the potential communication limitations between task regions. Moreover, a filtering strategy is proposed to quickly filter non-optimal task assignments to speed up the solving process.

To solve the task planning problem under individual and collaborative task requirements, we propose an algorithm to compute robots' execution strategies only based on each robot's local planning. The obtained execution strategies can still satisfy temporal and collaborative constraints of tasks. To further reduce robots' wait time in collaborations, we propose an execution strategy adjusting mechanism based on distributed communication, in which robots can locally adjust their execution strategies to reduce the wait time and optimize the total time cost.

We also provide theoretical analysis and simulation verification of the proposed framework in this paper. We prove the completeness of the framework under certain assumptions and analyze its time complexity and the factors that affect the optimality of robots' execution strategies. In addition, we conduct extensive simulation experiments under different numbers

中国知网 https://www.cnki.net

of robots, task types and environment scales to verify the scalability and efficiency of the proposed method, better than the conventional method based on global product automaton. The experimental results also verify the fast convergence of the iterative execution strategy adjusting mechanism and its optimization efficiency.

KeyWords: Multi-robot; task planning; linear temporal logic; distributed communication; task decomposition

中国知网 https://www.cnki.net

目录

致谢 I
摘要 II
Abstract III
目录 V
图目录 VIII
表目录 IX
1 绪论 1
1.1 课题背景与意义 1
1.2 多机器人任务规划研究现状 2
1.2.1 多机器人任务规划问题 2
1.2.2 时序约束下的任务规划 3
1.2.3 存在的问题 7
1.3 研究内容与章节安排 7
2 预备理论与问题描述 10
2.1 符号与含义 10
2.2 线性时序逻辑基本概念 11
2.2.1 线性时序逻辑 11
2.2.2 不确定性有限状态自动机 12
2.3 工作环境离散化建模与任务要求 14
2.4 问题描述 16
2.5 本章小结 18
3 时序约束下的协同任务分解与分配 19
3.1 基于自动机的时序任务独立性分解 19
3.1.1 分解状态识别 19
3.1.2 状态转移关系筛除 21
3.1.3 关键序列选择与分解 22
3.2 基于 SMT 模型的协同任务分配 22

中国知网 https://www.cnki.net

3.2.1 协同任务分配问题建模 23
3.2.2 任务分配方案的迭代求解与快速筛除 24
3.3 本章小结 25
4 多机器人执行策略的求解与优化 26
4.1 多机器人执行策略求解与执行 26
4.1.1 多机器人执行策略求解 26
4.1.2 协同状态识别与多机器人任务执行机制 28
4.2 基于混合整数规划的集中式执行策略优化 30
4.2.1 个体乘积自动机状态缩减 31
4.2.2 基于混合整数规划模型的执行策略优化 33
4.3 基于分布式迭代调整的执行策略优化 34
4.3.1 初始化 38
4.3.2 通信与迭代调整 39
4.3.3 终止条件 42
4.4 本章小结 42
5 多机器人时序任务规划算法的理论分析与实验验证 44
5.1 多机器人时序任务规划算法框架 44
5.2 完备性、复杂度与最优性分析 45
5.3 算法可行性与求解效率验证 47
5.3.1 多机器人时序任务规划框架的可行性 47
5.3.2 多机器人时序任务规划框架的可拓展性 48
5.3.3 执行策略迭代调整机制的求解效率与优化性能 49
5.3.4 任务分配方案的迭代评估与快速筛除效果 52
5.4 本章小结 53
6 结论与展望 54
6.1 本文的主要研究成果 54
6.2 下一步的研究方向 55
参考文献 57
攻读硕士学位期间的主要成果 62

中国知网 https://www.cnki.net

答辩委员会表决结果 63

中国知网 https://www.cnki.net

图目录

图 1.1 研究内容与章节安排 8
图 2.1 LTL_f 表达式 $(\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\ \mathrm{U}\ \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \rightarrow (\Diamond\pi^{ct_3})))$ 对应的 NFA 模型 13
图 2.2 机器人的工作环境与时序任务要求示例 15
图 3.1 图 2.1 对应的状态缩减后的 NFA 模型以及其中的分解状态 21
图 4.1 个体乘积自动机示意图 28
图 4.2 协同等待时间示意图 31
图 4.3 状态缩减后的个体乘积自动机 32
图 4.4 机器人执行策略迭代调整机制示意图 35
图 4.5 机器人执行策略迭代调整机制的优化效果 36
图 4.6 执行策略迭代调整机制的分布式通信机制 40
图 5.1 多机器人时序任务规划框架流程图 45
图 5.2 图 2.2 所示仿真案例中的机器人执行策略 47
图 5.3 不同数量的协同任务下 MILP 方法与迭代调整机制的性能比较 50
图 5.4 不同环境规模下 MILP 方法与迭代调整机制的性能比较 52
图 5.5 不同协同任务分配方案下的执行策略优化过程以及快速筛除策略的效果 52

中国知网 https://www.cnki.net

表目录

表 5.1　基于全局乘积自动机的方法（Baseline）对比多机器人时序任务规划算法（Proposed）..... 49
表 5.2　状态缩减时间、策略迭代调整机制求解时间以及 MILP 方法求解时间对比..... 51

中国知网　https://www.cnki.net

1 绪论

1.1 课题背景与意义

随着物联网的发展与智能设备的普及，多机器人之间的协作也开始广泛出现社会生产和生活的方方面面。在交通物流领域，行业开始推广地面智能导航车来进行社区快递的自动化送达，也有部分企业考虑使用多无人机系统来进行城市中货品的运送[1]，基于电动汽车的自动驾驶车辆也正在如火如荼的发展之中[2]，基于多种异构 AGV（Automated Guided Vehicles，自主导航车）的智能仓储系统也得到了广泛的实际应用[3-4]；在军事领域，各国都开始研发基于多种作战单元自主协同的军事系统[5]，如包含战斗、电子干扰、运输机等多机种的协同作战系统；在环境监测方面，运用地面车辆和空中无人机进行协同的电力线路巡检系统已经投入使用[6]，多 AUV 的协同探测也开始应用于海洋环境监测领域中[7]。在上述经济、社会、军事等领域的智能化发展过程中，多机器人的任务协同是其中的一项核心问题。

传统的机器人任务规划问题往往考虑机器人在工作环境中如何从起点顺利转移到终点，并搜索可行的路径。然而日益提高的社会智能化水平为机器人任务规划提出了更为复杂的任务要求，例如，在仓储物流环境中，某些物品的取放需要多种异构 AGV 的配合，比较重的货物的空中运输需要多架无人机的协同；在工厂环境中，某些商品的加工过程需要满足一定的次序约束；在环境监测领域，为了获得更为全面丰富的数据，可能需要携带不同种类传感器的机器人同时进行参数监测。在上面的场景中，多机器人系统需要处理的任务之间存在耦合关系，且具有时序上的约束，同时还需要考虑异构机器人的功能特点，这为多机器人系统的推广应用带来了挑战。

综上所述，研究在具有复杂时序约束下的多机器人任务协同机制，对推进多智能设备在社会生产生活中的应用具有重要的价值。而如何选择合理的建模方式来描述复杂时序约束下的机器人任务协同问题，并设计高效的任务规划算法来优化机器人的协同效率，是本文研究的重点问题。

中国知网 https://www.cnki.net

1.2 多机器人任务规划研究现状

1.2.1 多机器人任务规划问题

随着智能设备的发展和社会自动化水平的提升，基于多智能体理论的多机器人任务协同广泛存在于社会生产的方方面面。多机器人任务协同即求解系统中机器人的任务执行策略，以完成给定的任务要求，并进一步优化完成时间或者移动距离等指标。本文针对复杂任务约束下的多机器人任务规划问题，综合考虑机器人的任务协同和路径规划，包括协同任务的分配和协同关系的求解，以及在不同任务区域之间转移时机器人的路径搜索问题。

近年来，许多研究者针对多机器人任务规划问题提出了相应的求解算法。这些算法大致上可以分类为基于混合整数规划模型的算法，基于市场规则的算法和基于仿生理论的智能算法等[8]。从算法结构来看，可以分为集中式、分布式和混合式等三种结构。

（1）基于混合整数规划模型

混合整数规划（Mixed Integer Linear Programming, MILP）模型广泛应用于处理组合优化问题，例如旅行商问题、网络流问题、多目标追踪问题等。近年来的一些研究开始将整数规划模型应用于解决多机器人的组合优化问题中。混合整数规划模型可描述多种任务约束，且其求解过程具有一定的通用性，通常作为求解该类问题的基本方法[9-11]。文献 [12] 通过建立混合整数规划模型，来求解多机器人的路径搜索问题，并利用现有的求解器获得了良好的求解性能。文献 [13] 考虑了双臂机器人的任务执行序列的规划问题，通过建立混合整数规划模型来寻找每一个机械臂的最佳任务执行序列，从而优化总体的时间效率。文献 [14] 针对多 UAV（Unmanned Aerial Vehicle）搜索任务的规划问题，建立了集中式混合整数规划模型，并提出将复杂任务划分为简单子任务可以有效降低问题规模。但是，由于多机器人的任务规划问题是一个 NP 难问题，因此基于混合整数规划的完备算法在求解复杂度上随着机器人的规模与任务数量的增长呈指数增长，在实际应用中难以适用于大规模机器人系统的快速任务规划场景中。

（2）基于市场机制

基于市场的多机器人任务规划算法来源于人类社会中的市场竞拍机制，适用于多机器人分布式任务执行场景，可以有效减少通信依赖，提高机器人的自主性。在分布式多机器人控制领域，基于市场规则的协同算法具有广阔的前景[15-16]。文献 [17] 研究异构无

中国知网 https://www.cnki.net

人机对不同类型目标执行侦察、打击和评估任务的协同任务分配问题，作者设计了基于相邻局部通信的分布式拍卖算法，实现了多无人机协同任务分配问题的优化求解，并通过仿真证明了模型的合理性和求解方法的有效性。文献 [18] 利用改进合同网（Contact Net Protocol, CNP）算法求解了多机器人系统的最优任务分配问题，该算法利用区域信任度 RTD（Regional Trust Degree）有效减少了通信范围，并建立了一种层次化的结构来考虑任务约束和目标函数对规划结果的影响。在任务执行过程中，通过修改 RTD，算法可以根据环境变化实现自适应任务分配调整。在求解多机器人协同规划问题时，基于市场机制的算法具有易拓展性，快速收敛性等特点，尤其当机器人规模庞大，且环境存在动态变化时，基于市场机制的规划算法可以有效应对环境变化，而传统的求解算法则无法高效处理动态环境下的大规模机器人的任务规划问题。然而，基于市场机制的算法可以处理的任务类型较为简单，难以用于具有复杂时序约束的机器人任务规划问题中。

（3）基于智能优化算法

智能优化算法来自于人们对自然界中生物行为的观察，通过利用生物界的进化理论与群体协同理论来求解复杂度较高的数学问题。常用的智能优化算法有模拟退火算法、进化算法、禁忌搜索算法及群智能算法等，其中常用的群智能算法有蚁群算法和粒子群算法等。智能优化算法在求解复杂组合优化问题时，具有易于实现、适应性强、算法复杂度低等特点，因此被广泛应用到多机器人任务规划问题的求解中[19-21]。文献 [22] 利用蚁群优化算法求解了机器人的任务分配与轨迹规划问题，同时可以保证轨迹无碰要求以及任务之间的切换。文献 [23] 利用一种分支定界算法和遗传算法来求解多机器人的任务分配问题，并可以通过调整遗传算法的参数，权衡任务分配方案的计算时间与结果的优化程度。由于算法的随机性特点，智能优化算法一般无法对解的质量提供保证，且容易陷入局部最优解。

1.2.2 时序约束下的任务规划

随着社会生产的发展，多机器人系统需要处理的任务除了点到点的任务类型之外，还包括异构的具有不同功能的机器人的配合，以及一些复杂的时序任务。比如在智慧仓储系统中，需要具有不同机械臂终端的机器人相互配合来完成多种货物的搬运；在无人工厂中，某些产品的加工过程需要遵守严格的时序流程；在自主监测任务中，需要多种具有不同传感器的监测单元来对多个任务区域进行协同监控。在上述实际场景中，多机

中国知网 https://www.cnki.net

器人在执行任务时存在耦合关系，且不同任务的执行顺序可能存在一定的约束条件。为了解决上述具有耦合和时序约束的多机器人任务协同的问题，一部分学者开始将模型检查理论引入到机器人的控制与规划中。

基于模型检查理论的形式化方法，如线性时序逻辑（Linear Temporal Logic，LTL）语言，由于其简洁易懂的语法以及在描述时序约束中的表达能力，近年来吸引了学者的广泛关注[24-26]。线性时序逻辑原本是计算机软件系统模型检查中的一种命题逻辑，可以用来精确表述系统模型需要满足的一些时序特性，以便于对系统进行逻辑上的检查，有效处理并发场景下的死锁等情况，在航天器设计、银行交易系统等精密系统设计中存在广泛应用[27]。利用模型检查方法，我们可以利用线性时序逻辑表达式来表述具有时序约束的复杂任务要求，并通过进行模型检查过程来自动地获得满足任务要求的机器人执行策略，同时该过程也保证了机器人执行策略的可行性。

（1）单机器人时序任务规划

基于 LTL 的机器人任务规划，早先被用于单机器人的任务规划中，近十年来逐渐开始拓展到多机器人的任务协同中。早期的研究致力于将模型检查方法运用到机器人任务规划中，得到满足任务时序逻辑要求的可行解。通过利用现有的转换工具，一个时序逻辑表达式可以被转化为一个对应的自动机模型，借此可以直观地得到机器人的控制策略，并对机器人的行为有一定的安全性保证[28]。给定一个全局的 LTL 表达式，机器人可以通过构建一个融合了 LTL 表达式对应的自动机模型和机器人的工作环境模型的融合自动机，从而在其中得到自身的任务执行策略。文献 [29] 针对单机器人的复杂时序任务，利用线性时序逻辑表示机器人任务需求，并利用模型检查的方法得到满足任务要求的机器人执行策略。文献 [30] 提出了基于线性时序逻辑的单机器人复杂任务规划算法，该算法可以求得满足要求的最优任务序列，同时作者对于该算法的最优性进行了理论证明。

（2）多机器人时序任务规划

在多机器人任务规划领域，基于时序逻辑理论的相关研究可以分为两种模式，即自上而下（top-down）和自下而上（bottom-up），下面将分别进行介绍：

1）自上而下模式，基于全局时序逻辑表达式。

一方面，自上而下的研究模式通常假设给定多机器人系统一个全局的时序逻辑任务表达式，包含了多机器人系统需要完成的所有任务要求。机器人可以通过在一个全局乘

中国知网 https://www.cnki.net

积自动机上搜索一条接收路径进而得到自身的任务执行策略，该乘积自动机融合了所有机器人的环境模型以及全局时序逻辑表达式对应的自动机。文献 [31] 将文献 [30] 中提出的框架拓展到多机器人任务规划领域，将多机器人环境模型构建为一个联合环境模型，在乘积自动机上进行最优任务规划搜索，同时作者针对实际执行中运动时间的不确定性提出了鲁棒性规划算法。自上而下的规划算法的优势在于无需考虑全局任务的分配过程，通过构建一个全局乘积自动机模型，然后在乘积自动机上进行最优路径搜索即可得到每个机器人的最优任务规划。但它的缺点在于传统的图搜索方法需要基于完整的乘积自动机模型，而其中多机器人联合环境模型的状态空间随着机器人数量的增长而指数增长。另外，从 LTL 表达式转化到对应的自动机模型，其复杂度同样随着 LTL 表达式长度而呈指数增长。这些问题限制了集中规划算法的应用范围。

为了降低集中式多机器人时序任务规划算法的求解复杂度，研究人员提出了多种解决方法。文献 [32-33] 中尝试对机器人的加权切换系统进行状态缩减，并搜索满足任务要求的机器人执行策略。文献 [34-37] 利用自动机模型，对机器人的时序任务要求进行独立性分解，从而降低了机器人任务规划时的问题规模，但该框架只能用于解决有限时序任务，而无法处理无限巡回任务。文献 [38] 通过保守的独立性判别方式对表述任务的 STL(Signal Temporal Logic) 表达式进行拆分，在此基础上系统只需要求解更小规模的任务规划问题。

基于采样的状态空间搜索算法也获得了较多关注[39-40]。现有的文献中基于采样的时序任务规划方法大都借鉴于机器人路径规划领域的经典采样算法，如概率路线图（Probabilistic Road Maps），快速随机搜索树算法（Rapidly-exploring Random Trees）等。其中，RRT 算法由于可以满足更加一般化的环境结构而得到广泛应用[41]。文献 [42] 针对单机器人时序任务规划问题，将机器人的连续环境模型通过采样增量式地构建为克里普克结构（Kripke Structure），避免了对环境进行无差别采样而引起的状态增长。但这种方法随着采样次数的增长，环境图模型会变得逐渐密集，引起搜索时间的增长，因此不适用于大规模时序任务规划问题。文献 [43] 提出基于采样的方式增量式地构建机器人切换系统模型，但仍具有文献 [42] 中方法的限制。上述文献都是针对连续环境，利用基于采样的方法构建机器人在工作环境中的状态转移图模型。而文献 [44] 则在给定单机器人环境模型的情况下，针对多机器人联合环境模型的状态空间提出基于采样的搜索算法。作者基于 RRT* 算法[45]的思路，在乘积自动机的状态空间中通过采样，增量式地构造一棵

中国知网 https://www.cnki.net

从初始节点到接收节点的生成树，并证明了所提出的基于采样的乘积自动机状态空间搜索算法具有概率完备和渐近最优的特性，并且具备良好的可拓展性。后续的研究进一步拓展了该方法的求解效率[46-48]。

另外，有一些学者基于强化学习方法研究了机器人时序任务规划问题，以避免进行工作空间离散化[49-51]。还有一部分工作将时序逻辑任务规划问题转化为混合整数规划模型，然后利用现有的求解器如 Gurobi 来求解[52-53]。

2）自下而上模式，基于个体时序逻辑表达式。

另一方面，自下而上的规划方法倾向于将任务表达式分配到个体机器人，通过个体机器人之间的分布式协作来共同完成全局的任务要求[54-55]。M. Guo 等人研究了多个机器人的某些个体任务之间存在约束的协同问题[56]。该论文中机器人首先独立地求解一个离线的任务执行策略，随后在实际任务执行中，通过在线的通信与任务分配来完成某些协同动作。上述算法在 [57] 中得到了进一步改进，包括考虑了异构机器人的情况以及在线的任务交换机制。J. Tumova 研究了与文献 [56] 略有不同的问题设定，其中每个机器人有自身的个体时序逻辑任务表达式，包括一个独立的区域移动任务表达式和依赖于与其它机器人协同的协同时序逻辑任务表达式[58]。该论文提出了一种两阶段的基于自动机的方法来解决上述问题。首先，各个机器人独立在本地构建一个本地乘积自动机，来获得自身的运动规划。这一本地乘积自动机将会被进一步缩减状态，即移除与协同任务无关的状态。然后，该方法构造一个全局乘积自动机，融合了所有机器人进行状态缩减后的本地乘积自动机。通过在该全局乘积自动机上搜索路径即可得到机器人之间的协同策略。尽管在状态缩减阶段，机器人的本地乘积自动机的规模大幅减小，但该方法随着机器人数量的增长依然具有指数复杂度。Y. Kantaros 研究了与本文类似的问题，即个体机器人具有它们各自的个体任务时序逻辑表达式，同时它们需要协同完成一个全局给定的协同任务表达式[59]。然而，文献 [59] 中的协同任务表达式仅限于持续通信任务，即机器人持续地与其它具有协同关系的机器人进行会合与通信。文中提出了一种在线式的优化方法，来迭代优化机器人会合的位置，从而不断降低机器人总的移动距离。上述基于自下而上模式的相关文献中，普遍假设协同任务预先被完全或部分地分配给机器人。

中国知网 https://www.cnki.net

1.2.3 存在的问题

之前的研究工作中，给定机器人个体任务表达式的多机器人任务规划方法依赖于分布式的在线规划或集中式的计算。许多在线算法需要假设协同任务要求被集成在个体任务表达式中，即协同任务被完全或者部分地预先分配给了机器人。另外，在线规划算法往往无法从全局的角度优化机器人的实际任务执行效率。基于集中式计算的方法可以适用于更加一般化的任务约束，但是其求解复杂度随着机器人规模的增大而呈指数增长，因此可拓展性较差。

此外，现有的时序逻辑任务规划算法大都着眼于如何获得可行并且在移动距离上较优的机器人执行策略，而对于多机器人执行任务时的时间效率的优化则考虑较少。时间效率优化问题在时序任务的协同中尤其显著，因为一些任务可能需要多个机器人同步执行，这会造成额外的协同等待时间，从而降低多机器人系统的执行效率。之前的研究工作尚未充分考虑对于时序逻辑约束下的多机器人协同效率的优化问题。

1.3 研究内容与章节安排

基于上述多机器人时序任务规划领域的发展现状以及存在的问题，本文将聚焦于研究时序约束下的机器人协同任务的分解与分配，基于本地计算的机器人执行策略高效求解，以及多机器人任务协同效率的优化等科学问题。本文的研究内容如图 1.1 所示。各章节主要研究内容具体为：

第二章：预备理论与问题描述。本章介绍预备理论知识，包括线性时序逻辑语言的基本概念以及其与自动机模型的对应转化关系，机器人的区域转移和任务执行能力的描述以及工作环境的离散化建模。最后根据实际应用中的任务场景，利用线性时序逻辑语言描述多机器人的复杂时序任务要求，并给出本文研究的时序逻辑约束下的多机器人任务协同问题的形式化描述。

第三章：时序约束下的协同任务分解与分配。针对时序约束下的任务协同问题存在建模难，求解复杂度高等问题，本章基于协同任务时序逻辑表达式对应的自动机模型，对协同任务要求进行独立性分解，从而简化协同任务之间的耦合关系。为进一步将任务规划过程分摊到个体机器人以降低整体规划问题的规模，本章建立了 SMT 模型来求解协同任务分配问题，并考虑了通信受限时如何满足协同任务间的时序约束。该方法利用

中国知网 https://www.cnki.net

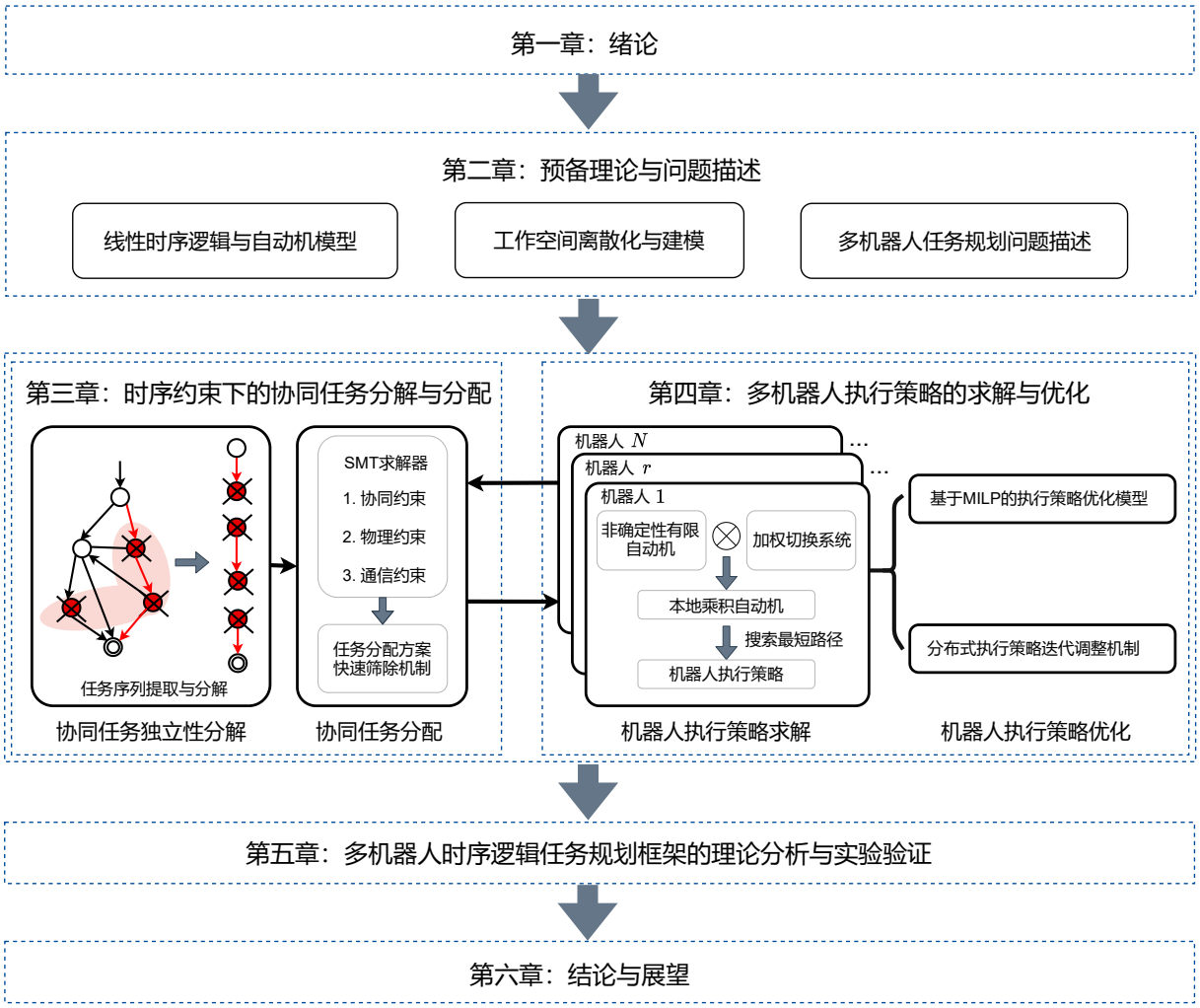


图 1.1: 研究内容与章节安排

SMT 模型的迭代求解机制提出一种快速筛除策略，实现对协同任务分配方案的迭代求解与快速筛除。

第四章：多机器人执行策略的求解与优化。本章在第三章协同任务分配的基础上，针对机器人需要同时处理时序约束下的个体任务要求和被分配的协同任务要求的复杂任务规划问题，提出一种由个体机器人在本地计算的执行策略求解方法，将计算量分摊到每个机器人，从而提高了任务规划的效率。进一步，针对机器人参与协同任务时由于同步要求而产生的低效的协同等待时间，本文提出一种基于分布式通信的机器人执行策略迭代调整机制，以优化机器人执行协同任务的时间效率，从而降低多机器人系统的总时间损耗。通过与基于混合整数规划模型的执行策略优化方法对比，验证了该迭代调整机制的求解快速性与良好的优化效果。

第五章：多机器人时序任务规划算法的理论分析与实验验证。本章结合第三章和第

中国知网 https://www.cnki.net

四章中的研究成果，提出用于复杂时序任务约束下的多机器人任务规划框架。随后，本章分析了该规划框架的完备性，算法的计算复杂度，并讨论了影响执行策略最优性的因素。在不同的任务类型、环境规模以及机器人数量下，本章对比了所提出的多机器人时序逻辑任务规划框架与常用的基于全局乘积自动机的经典规划算法的性能表现，验证了该框架的可行性，在求解效率方面的显著优势以及执行策略的性能表现。另外，本章还对比了本文所提出的执行策略迭代调整机制与基于混合整数规划模型的策略优化方法的求解效率与优化程度，验证该迭代调整机制在求解效率方面的突出优势以及良好的优化效果。

第六章总结了本文的主要研究成果与贡献，并进一步分析了本文将来的研究方向，包括动态环境下的多机器人任务规划，考虑机器人动力学约束的控制器设计以及在具有巡回任务约束下的多机器人协同优化问题等。

中国知网 https://www.cnki.net

2 预备理论与问题描述

本章中，我们将首先介绍用于描述多机器人的时序任务要求的线性时序逻辑语言，包括其基本语法和该逻辑语言对应的自动机模型。随后，我们根据机器人在工作环境中的区域转移能力和可提供的技术功能，建立机器人在环境中的加权切换系统模型。最后根据用户给定的时序任务要求，提出多机器人系统的执行策略求解与优化问题。

2.1 符号与含义

符号清单、缩写	含义
ϕ	协同任务的有限线性时序逻辑表达式
φ_r	机器人 r 的个体任务的有限线性时序逻辑表达式
$\mathcal{F}$	非确定性有限自动机 (NFA)
ρ_F	非确定性有限自动机上的一条路径，由自动机中相邻状态组成
$\mathcal{AP}$	原子命题集合
π	单个原子命题
σ	命题序列，由原子命题的集合所组成的序列
$\mathcal{W}$	工作环境离散化后的图模型
$\mathcal{N}$	所有机器人的编号集合
r	机器人 r 的编号
$\overline{\mathbb{T}}_r$	机器人 r 的个体任务集合，每项个体任务可由机器人 r 独自完成
$\widetilde{\mathbb{T}}$	协同任务集合，每项协同任务要求不少于一个机器人协同完成
wTS_r	机器人 r 的加权切换系统，反映其区域转移能力和功能特点
T_r^{colla}	机器人 r 完成自身任务要求的总时间损耗（考虑协同等待时间）
T^{colla}	所有机器人 $r \in \mathcal{N}$ 完成所有任务的时间损耗之和
LTL	Linear Temporal Logic，线性时序逻辑
NFA	Nondeterministic Finite Automaton, 非确定性有限自动机
SMT	Satisfiability Module Theories，可满足性模理论
MILP	Mixed Integer Linear Programming，混合整数线性规划

中国知网 https://www.cnki.net

2.2 线性时序逻辑基本概念

2.2.1 线性时序逻辑

线性时序逻辑（LTL）是用来描述原子命题（Atom Proposition，以下简称命题）之间的时序约束的一种形式化语言，可以表述命题之间的多种时序约束。在本文中，我们使用线性时序逻辑来表达机器人需要执行的任务之间的复杂时序约束。在机器人的任务规划中，原子命题用来指示机器人执行任务的状态。例如，考虑在仓储环境中，某个机器人需要去区域 A 交付商品，在此之前需要去区域 B 提取商品，则我们可以将“去区域 A 交付商品”和“去区域 B 提取商品”分别定义为原子命题 π^A 和 π^B。当机器人成功在区域 B 提取了商品时，原子命题 π^B 就为真命题，而当机器人在区域 A 完成交付后，原子命题 π^A 即为真。上述原子命题 π^A 和 π^B 之间的时序约束，可以用线性时序逻辑语言表述为 $\Diamond(\pi^B \wedge (\Diamond\pi^A))$。关于线性时序逻辑语言的具体语法介绍如下：

一个 LTL 表达式 φ 定义在原子命题集合 $\mathcal{AP}$ 上，并由时序运算符和逻辑运算符相连接，其构成定义为[27]：

$$\varphi ::= \text{true} \mid \pi \mid \varphi_1 \wedge \varphi_2 \mid \neg\varphi \mid \bigcirc\varphi \mid \varphi_1 \mathrm{U} \varphi_2,$$

其中 true 为恒真命题，$\pi \in \mathcal{AP}$ 是一个原子命题。另外两种常见的时序运算符 $\Diamond$ (eventually，在将来某个时刻) 和 $\Box$ (always，现在以及以后) 可以根据上述已有的运算符 U (until) 推导得出，即 $\Diamond\varphi := \text{true}\ \mathrm{U}\ \varphi$，$\Box\varphi := \neg\Diamond\neg\varphi$。

在解释上述线性时序逻辑表达式的语法之前，我们首先定义一个 LTL 表达式的接收语言。假设 φ 是在原子命题集合 $\mathcal{AP}$ 上定义的 LTL 表达式，则这个表达式的接收语言集合表示为：

$$Word(\varphi) = \{\sigma \in (2^{\mathcal{AP}})^\omega | \sigma \vDash \varphi\}$$

其中关系符号 $\vDash \subseteq (2^{\mathcal{AP}})^\omega \times LTL$，表示序列 σ 可以满足 LTL 表达式 φ 的要求，序列 σ 是由集合 $\mathcal{AP}$ 中的原子命题的子集所构成的无限长序列。

对于 $\sigma = \sigma(0)\sigma(1)\sigma(2)\cdots \in (2^{\mathcal{AP}})^\omega$，$\sigma[j...] = \sigma(j)\sigma(j+1)\cdots$ 表示序列 σ 从第 $j+1$ 个状态 $\sigma(j)$ 开始的后缀序列。下面根据序列与 LTL 表达式之间的满足关系，对 LTL 表达式进行语法上的解释：

中国知网 https://www.cnki.net

- $\sigma \vDash true$；
- $\sigma \vDash \pi$，当且仅当 $\pi \in \sigma(0)$；
- $\sigma \vDash \varphi_1 \wedge \varphi_2$，当且仅当 $\sigma \vDash \varphi_1$ 且 $\sigma \vDash \varphi_2$；
- $\sigma \vDash \neg\varphi$，当且仅当 σ 不满足 φ；
- $\sigma \vDash \bigcirc\varphi$，当且仅当 $\sigma[1...] = \sigma(1)\sigma(2)\cdots \vDash \varphi$；
- $\sigma \vDash \varphi_1 \mathrm{U} \varphi_2$，当且仅当 $\forall\, 0 \leq i < j, \exists\, j \geq 0, \sigma[j...] \vDash \varphi_2$，且 $\sigma[i...] \vDash \varphi_1$；
- $\sigma \vDash \Diamond\varphi$，当且仅当 $\exists\, j \geq 0, \sigma[j\cdots] \vDash \varphi$，其中 $\Diamond\varphi := \mathrm{true}\mathrm{U}\varphi$；
- $\sigma \vDash \Box\varphi$，当且仅当 $\forall\, j \geq 0, \sigma[j\cdots] \vDash \varphi$，其中 $\Box\varphi := \neg\Diamond\neg\varphi$；
- $\sigma \vDash \Box\Diamond\varphi$，当且仅当 $\forall\, i \geq 0 \exists\, j \geq i, \sigma[j\cdots] \vDash \varphi$；
- $\sigma \vDash \Diamond\Box\varphi$，当且仅当 $\exists\, i \geq 0 \forall\, j \geq i, \sigma[j\cdots] \vDash \varphi$。

利用线性时序逻辑，我们可以表示许多种类型的任务，如：安全性任务（如 $\Box\neg\varphi$，表示在整个任务执行过程中避免 φ 被满足）；时序性任务（如 $\Diamond(\varphi_1 \wedge \Diamond(\varphi_2 \wedge \Diamond\varphi_3))$，表示命题 φ_1，φ_2，φ_3 需要按其先后顺序被满足）；反应性任务（如 $\varphi_1 \rightarrow \Diamond\varphi_2$，表示命题 φ_1 被满足意味着命题 φ_2 也终将被满足）；巡回任务（如 $\Box\Diamond\varphi$，表示命题 φ 将无限次被重复满足）等。这体现了线性时序逻辑语言强大的任务描述能力，因而可以满足多机器人协同场景中的任务描述需求。

在本文中，我们利用一种称为有限 LTL（finite LTL）的语言[34, 60]来表述机器人任务之间的时序约束，记为 LTL_f。LTL_f 表达式是 LTL 的一种特殊类型，使用与 LTL 相同的语法。不同之处在于，LTL_f 表达式可以被有限长的命题序列所满足，即对于 LTL_f 表达式 φ 而言，其对应的接收语言为：$Word(\varphi) = \{\sigma \in (2^{\mathcal{AP}})^* | \sigma \vDash \varphi\}$。因此，对于 LTL_f 表达式而言，$\Box$（always）的作用范围不是无限长的序列，而是在有限序列的范围内。

另外，我们从语法中取消了 $\bigcirc$ (next)，这是因为时序运算符 $\bigcirc$ 的作用是区分命题序列中某个命题重复为真的次数。然而在实际场景中，使一个命题为真对应于机器人连续的动作，从中区分该命题为真的次数不具有现实意义，类似的设定在文献 [61] 中也可以看到。关于 LTL 语法的详细内容可参阅文献 [27]。

2.2.2 不确定性有限状态自动机

给定一个 LTL_f 表达式 φ，我们可以利用现有的工具 [62-63] 将其转化为一个对应的不确定性有限自动机（Nondeterministic Finite Automaton，NFA），见定义2.1。

中国知网 https://www.cnki.net

定义 2.1 (不确定性有限自动机). 一个不确定性有限自动机 $\mathcal{F}$ 可以表示为一个元组 $\mathcal{F} := \langle \mathcal{Q}_F, \mathcal{Q}_F^0, \alpha, \delta, \mathcal{Q}_F^F \rangle$，其中：

- $\mathcal{Q}_F$ 是一个有限的状态集合；
- $\mathcal{Q}_F^0 \subseteq \mathcal{Q}_F$ 是初始状态的集合；
- α 是定义在原子命题集合 $\mathcal{AP}$ 上的布尔表达式集合；
- $\delta : \mathcal{Q}_F \times \mathcal{Q}_F \to \alpha$ 表示了 $\mathcal{F}$ 中相邻状态之间的转移条件；
- $\mathcal{Q}_F^F$ 是接收状态的集合。

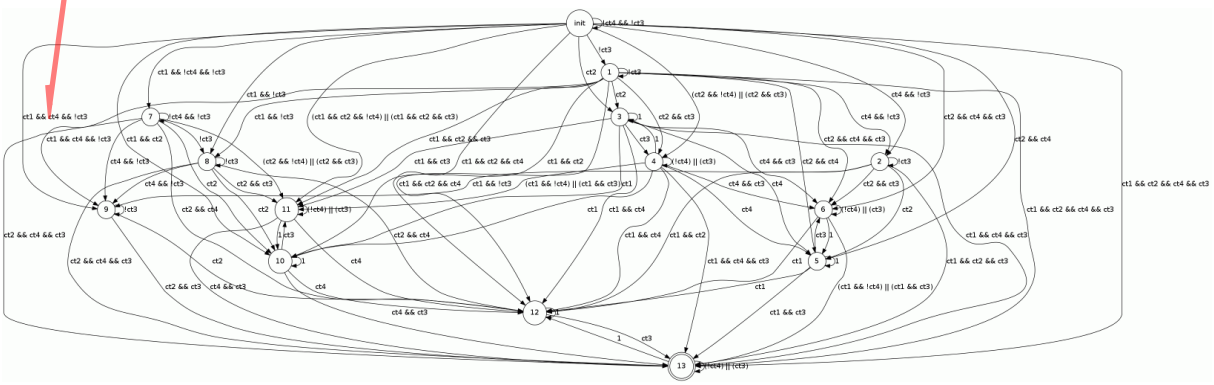


图 2.1: LTL_f **表达式** $(\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\ \mathrm{U}\ \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \to (\Diamond\pi^{ct_3})))$ **对应的 NFA 模型**

图 2.1 展示了 LTL_f 表达式 $(\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\ \mathrm{U}\ \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \to (\Diamond\pi^{ct_3})))$ 对应的 NFA 模型，该表达式对应于下一小节中图 2.2 中的 ϕ，由转化工具 [62] 得到。该表达式的含义为：机器人需要执行任务 ct_1（使任务 ct_1 对应的原子命题为真），ct_2 以及 ct_4；在执行任务 ct_2 之前，不可以执行任务 ct_3；若执行 ct_4，则要求在未来某个时刻任务 ct_3 也被执行。有关工作环境中任务的定义将在下一节中给出。

由 NFA $\mathcal{F}$ 中的状态 $q_F \in \mathcal{Q}_F$ 所组成的一个状态序列，若满足 $\forall\, i \in [1 : L-1], (\rho_F(i), \rho_F(i+1)) \in \delta$，则将其称为 $\mathcal{F}$ 中的一条路径（run），记为 ρ_F，其中 $L = |\rho_F|$ 为路径 ρ_F 的长度，$[1 : L-1]$ 代表了一个从 1 开始以步长 1 递增到 $L-1$ 的下标集合。一条路径 ρ_F 被称为接收路径（accepting）当且仅当该序列从 $\mathcal{F}$ 中的某个初始状态 $q_F^0 \in \mathcal{Q}_F^0$ 出发，终止于某个接收状态 $q_F^F \in \mathcal{Q}_F^F$。给定一个有限命题序列 $\sigma = \sigma(1)\sigma(2)\ldots\sigma(L-1)\sigma(L)$，若满足 $\forall\, r \in [1 : L-1], \sigma(i) \vDash \delta(\rho_F(i), \rho_F(i+1))$，则我们称 σ 描述了（describe）一条路径 ρ_F。其中，$\sigma(i)$ 是原子命题或其非命题的集合，可以使得布尔表达式 $\delta(\rho_F(i), \rho_F(i+1))$ 为真。

中国知网 https://www.cnki.net

一个命题序列 σ 满足 LTL_f 表达式 φ 当且仅当至少存在一个由命题序列 σ 所描述的路径为 φ 所对应的 NFA 的接收路径。另一方面，给定任意一个 LTL_f 表达式 ϕ 对应的 NFA 中的接收路径 ρ_F，根据 ρ_F 中相邻状态在 NFA 中的状态转移条件而获得的命题序列 σ，一定可以满足 LTL_f 表达式 ϕ。

在 NFA 上描述一条路径的最少命题序列，称为该路径的关键序列（Essential Sequence）[34]，见定义 2.2。

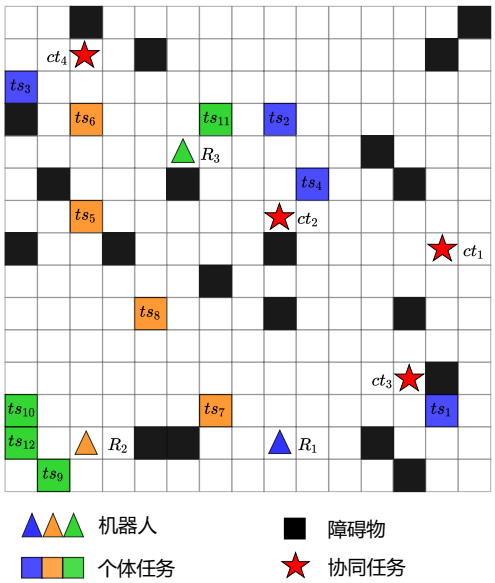
定义 2.2 (关键序列)**.** 对于一个原子命题序列 $\sigma = \sigma(1)\ldots\sigma(L)$，它被称为“关键序列”当且仅当其满足：（1）$\sigma$ 描述了 $\mathcal{F}$ 中的一条路径 ρ_F；（2）$\forall\, i \in [1:L-1], \pi \in \mathcal{AP}$，$\sigma(i)\backslash\{\pi\} \nvDash \delta(\rho_F(i), \rho_F(i+1))$。

由定义 2.2 可知，对于描述 $\mathcal{F}$ 中路径 ρ_F 的关键序列 σ 而言，若命题序列 σ 任何一个下标 i 处的命题集合 $\sigma(i)$ 中的任何一个命题不为真，则无法满足对应的 $\mathcal{F}$ 中的状态转移条件，也即关键序列 σ 中的任何一个命题对于路径 ρ_F 的描述都是“关键”的。上述命题序列可以直接通过观察 $\mathcal{F}$ 中的状态转移条件获得，这是由于 NFA 中的状态转移条件都表示为或与表达式的形式（Conjunctive Normal Form，CNF），其形式为 $\bigvee \wedge \pi$。例如，对图 2.1 中状态 $init$ 到状态 2 的转移条件 $ct_4 \wedge \neg ct_3$ 而言，提取出的关键序列即为 $\{ct_4\}$。注意，在此我们忽略了转移条件中的非命题，这是因为在后文的定义中，非命题的实际含义为不执行该命题对应的任务。实际中机器人不执行某一项任务和直接忽略对应的非命题是等价的。

2.3 工作环境离散化建模与任务要求

考虑多机器人集合 $\mathcal{N} := \{1, ..., N\}$ 在工作环境中工作，如图 2.2 所示，图中展示了在含有 3 个机器人（彩色三角形）的仿真实例中机器人的工作环境，以及机器人具有的 LTL_f 任务表达式。通过对机器人的工作环境进行离散化处理，例如在图 2.2 中进行的栅格化划分，我们可以将工作环境中的任务区域（彩色正方形和红色星形区域）以及这些区域之间的转移关系建模为一个图模型 $\mathcal{W} = \langle \mathcal{Q}, \mathcal{E} \rangle$。其中，$\mathcal{Q}$ 为工作环境中的任务区域的集合，在某个区域 $q \in \mathcal{Q}$ 中，机器人可以在该任务区域中根据自己可提供的技术功能进行相应的动作，如使用机械臂抓取环境中的货物，或调用摄像头拍摄环境照片等等；$\mathcal{E}$ 包含了机器人在这些任务区域之间的转移关系，其中 $\forall\, q_i, q_j \in \mathcal{Q}$，$(q_i, q_j) \in \mathcal{E}$ 表

中国知网 https://www.cnki.net



$$\varphi_1 = (\Diamond(\pi^{ts_1} \vee \pi^{ts_4})) \wedge (\Diamond\pi^{ts_2}) \wedge (\neg\pi^{ts_3} \text{ U } \pi^{ts_2}) \wedge (\Box(\pi^{ts_4} \rightarrow (\Diamond\pi^{ts_3})))$$

$$\varphi_2 = (\Diamond(\pi^{ts_5} \vee \pi^{ts_8})) \wedge (\Diamond\pi^{ts_6}) \wedge (\neg\pi^{ts_7} \text{ U } \pi^{ts_6}) \wedge (\Box(\pi^{ts_8} \rightarrow (\Diamond\pi^{ts_7})))$$

$$\varphi_3 = (\Diamond(\pi^{ts_9} \vee \pi^{ts_{12}})) \wedge (\Diamond\pi^{ts_{10}}) \wedge (\neg\pi^{ts_{11}} \text{ U } \pi^{ts_{10}}) \wedge (\Box(\pi^{ts_{12}} \rightarrow (\Diamond\pi^{ts_{11}})$$

$$\phi = (\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3} \text{ U } \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \rightarrow (\Diamond\pi^{ct_3})))$$

图 2.2: 机器人的工作环境与时序任务要求示例

明机器人可以在任务区域 q_i 和 q_j 之间移动。需要注意的是，由于机器人的外形和功能可能是不同的，因此每个机器人在这些任务区域之间的转移能力是不同的。我们将机器人 r 可提供的技术功能表示为 c_j，机器人通过执行该技术功能可以独自或与其它机器人配合完成工作环境中的任务，这里 $c_j \in Cap$，集合 $Cap := \{c_j\}_{j\in\{1,\ldots,|Cap|\}}$ 包含了所有可由机器人提供的技术功能。我们用集合 $\mathcal{N}_j := \{r \mid \text{机器人 } r \text{ 具备功能 } c_j\}$ 来表示所有具备功能 c_j 的机器人集合。

工作环境中分布着多项需要机器人完成的任务，它们的集合表示为 $\mathbb{T}$。机器人完成任务需要满足的条件见定义 2.3。

定义 2.3 (任务要求)**.** 环境中的任务可以表示为一个元组 $ts := \langle \pi^{ts}, \{(c_j, m_j^{ts})\}_{j\in I_{ts}}, q^{ts}\rangle$，其中：

- π^{ts} 是任务 ts 所对应的唯一的原子命题;
- $q^{ts} \in \mathcal{Q}$ 对应于任务 ts 所在的任务区域;
- $\{(c_j, m_j^{ts})\}_{j\in I_{ts}}$ 表示若要满足任务 ts 的完成条件，即 π^{ts} 为真，当且仅当 $\forall j \in I_{ts}$，至少 m_j^{ts} 个具有功能 c_j 的机器人被同时部署在任务区域 q^{ts} 中并执行自身的相应功能。这里 $I_{ts} \subseteq \{1, \ldots, |Cap|\}$ 表示完成任务 ts 所需的所有机器人的功能类型的集合.

这个集合对于单个机器人个体任务也是同样适用的？

机器人 r 在工作环境 $\mathcal{W}$ 中的区域移动能力和可提供的技术功能可以建模为一个加权切换系统（weighted Transition System, wTS），见定义 2.4。

中国知网 https://www.cnki.net

定义 2.4 (加权切换系统). 机器人 r 的加权切换系统 wTS_r 可以表示为一个元组 $wTS_r = \langle \mathcal{Q}_r, q_r^0, \rightarrow_r, \omega_r, \mathcal{AP}_r, L_r \rangle$，其中：

- $\mathcal{Q}_r \subseteq \mathcal{Q}$ 是一个有限的状态集合，为集合 $\mathcal{Q}$ 的子集，其中每个状态对应于工作环境 $\mathcal{W}$ 中的任务区域；
- q_r^0 代表初始状态，即机器人 r 在初始时刻所处的区域；
- $\rightarrow_r \subseteq \mathcal{E}$ 包括了对机器人 r 而言可行的状态转移，即若 $(q_r^i, q_r^j) \in \rightarrow_r$，表明从区域 q_r^i 到区域 q_r^j 之间存在一条可行的转移路径；
- $\omega_r : \mathcal{Q}_r \times \mathcal{Q}_r \rightarrow \mathbb{R}_{\geq 0}$ 是一个权重函数，代表机器人 r 在两个任务区域之间转移所花费的时间；在任务规划层面不考虑机器人之间的相互避障
- $\mathcal{AP}$ 是工作环境 $\mathcal{W}$ 分布的任务对应的原子命题集合；
- $L_r : \mathcal{Q}_r \rightarrow 2^{\mathcal{AP}}$ 是一个标签函数，满足 $\forall\, ts \in \mathbb{T}, \pi^{ts} \in L_r(q^{ts})$ 当且仅当 $\exists\, j \in I_{ts}$，$r \in \mathcal{N}_j$。

机器人 r 的执行策略 τ_r 即为 wTS_r 上的一条路径，由 wTS_r 中连续的状态组成。该执行策略 τ_r 包括了机器人 r 在工作环境中转移的路径，以及在任务区域的执行计划。当机器人根据 τ_r 中的顺序，在状态对应的任务区域之间转移，并且在对应的任务区域独自，或与其它机器人的协同执行动作，即可完成区域内的任务。根据标签函数 L_r 的定义，τ_r 的执行过程会产生一个由任务对应的原子命题集合组成的序列 σ，其中原子命题取值为真，且 $\sigma \in \{2^{\mathcal{AP}}\}^*$。

2.4 问题描述

在社会生产实际中，存在诸如：机器人需要去区域 A 抓取某一商品，并在另一项任务 C 结束前，把这一物品送到区域 B。在第 2.3 节中，我们定义了完成工作环境中的单个任务需要满足的条件，包括对机器人数量和类型的要求。本小节中，我们将进一步利用第 2.2 节中介绍的线性时序逻辑语言，来描述工作环境中的任务之间的时序约束，并提出本文研究的主要科学问题。

在本文中，我们考虑如下的问题，即（1）每个机器人 $r \in \mathcal{N}$ 在工作环境中需要独立完成某些个体任务，这些个体任务之间的时序约束用 LTL_f 表达式 φ_r 来表述，称

中国知网 https://www.cnki.net

为机器人 r 的个体任务 LTL_f 表达式，其中 φ_r 定义在机器人 r 的个体任务所对应的命题集合 $\overline{\mathbb{T}}_r$ 上。$\forall\, ts \in \overline{\mathbb{T}}_r$，有 $|I_{ts}| = 1, m_j^{ts} = 1$，且 $r \in \mathcal{N}_j$；（2）多个机器人需要协同完成工作环境中的某些协同任务，这些协同任务之间的时序约束用一个全局协同任务 LTL_f 表达式 ϕ 来表述，该表达式定义在协同任务对应的命题集合 $\widetilde{\mathbb{T}}$ 上，其中每项协同任务 $ct \in \widetilde{\mathbb{T}}$ 可能要求不少于一个具有不同功能的机器人的协同才能完成，见定义 2.3。需要特别注意的是，为了区分上述两种任务，即个体任务和协同任务，在后文中我们特别使用 ct 来表示集合 $\widetilde{\mathbb{T}}$ 中的某项协同任务，而集合 $\bigcup_{r\in\mathcal{N}} \overline{\mathbb{T}}_r$ 中的个体任务沿用 ts 来表示。例如，在图 2.2 所示的仿真案例中，机器人 1 的个体任务 LTL_f 表达式为 $\varphi_1 = (\Diamond(\pi^{ts_1} \vee \pi^{ts_4})) \wedge (\Diamond\pi^{ts_2}) \wedge (\neg\pi^{ts_3}\ \mathrm{U}\ \pi^{ts_2}) \wedge (\Box(\pi^{ts_4} \rightarrow (\Diamond\pi^{ts_3})))$，而所有机器人的全局协同任务 LTL_f 表达式为 $\phi = (\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\ \mathrm{U}\ \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \rightarrow (\Diamond\pi^{ct_3})))$。

假设 2.1. 我们假设个体任务 LTL_f 表达式 φ_r 的执行过程是相互独立的，即某个机器人执行自身的 φ_r 时并不会影响其它机器人的状态。另外，我们假设个体任务 LTL_f 表达式 φ_r 的执行过程和协同任务 LTL_f 表达式 ϕ 的执行过程也是相互独立的。上述假设的形式化表述为：$\forall\, i, j \in \mathcal{N}$，$i \neq j$，$\overline{\mathbb{T}}_i \cap \overline{\mathbb{T}}_j = \emptyset$，且 $\forall\, r \in \mathcal{N}$，$\widetilde{\mathbb{T}} \cap \overline{\mathbb{T}}_r = \emptyset$。

这一假设在实际任务场景中是合理的，因为若存在部分个体任务和协同任务之间不是相互独立的，则我们可以将这些不独立的任务要求都添加到协同任务表达式 ϕ 中，从而使上述假设满足。

需要说明的是，若仅给定多机器人系统的全局协同任务 LTL_f 表达式，上述任务规划问题即为经典的自上而下式的多机器人时序逻辑任务规划问题；若仅给定每个机器人的个体任务 LTL_f 表达式，则该任务规划问题即为自下而上式的多机器人协同问题。本文中同时考虑了机器人具有上述两种任务约束的情况，从而产生了新的问题，即机器人如何在满足个体任务 LTL_f 表达式的同时，与其它机器人进行合理高效的协同，以保证全局协同任务 LTL_f 表达式也能被满足，同时尽可能降低多机器人系统的时间损耗。

当给定每个机器人 r 的执行策略 τ_r 时，每个机器人可以并行地异步执行 τ_r，在到达任务区域时独自执行个体任务，或在协同任务区域按照任务要求等待与其它机器人的协同。机器人完成所有任务要求的总时间定义为 $T^{\mathrm{colla}} = \sum_{r\in\mathcal{N}} T_r^{\mathrm{colla}}$，其中 T_r^{colla} 表示机器人 r 执行策略 τ_r 所花费的总时间。其中，$T_r^{\mathrm{colla}} = T_r^{\mathrm{indiv}} + delay_r$，$T_r^{\mathrm{indiv}}$ 表示在不考虑与其它机器人的协同的情况下，机器人 r 按照执行策略 τ_r 完成所有的区域转移所

中国知网 https://www.cnki.net

消耗的时间，而 $delay_r$ 表示机器人 r 在与其它机器人合作完成协同任务时所产生的协同等待时间（由于不同的机器人到达同一协同任务区域的时间不同而导致）。我们假设当某个协同任务要求的机器人数量和种类得到满足时，该协同任务即可立即结束，对应的原子命题为真。换言之，相较机器人在不同任务区域之间的转移时间，其在任务区域内的任务执行时间可以忽略不计。对于任务执行时间不可忽略的情况，我们可以将该执行时间等价地转换到机器人的区域转移时间中。

本文研究的多机器人任务规划问题定义如下：

问题 1. 考虑集合 $\mathcal{N}$ 中的机器人在工作环境 $\mathcal{W}$ 中运行，并假设每个机器人 r 在工作环境中的区域转移能力和可提供的技术功能被建模为一个加权切换系统 wTS_r。给定每个机器人 r 的个体任务 LTL_f 表达式 φ_r，以及给定所有机器人一个全局协同任务 LTL_f 表达式 ϕ，在假设 2.1下，求解每个机器人 r 在加权切换系统 wTS_r 中的执行策略 τ_r，使得：

(1) $\forall\, r \in \mathcal{N}$，机器人 r 按照执行策略 τ_r 在工作区域之间移动并执行自身的技术动作时，所生成的原子命题序列可以满足个体任务 LTL_f 表达式 φ_r；

(2) 当所有机器人执行自身的执行策略时，所生成的原子命题序列可以满足全局协同任务 LTL_f 表达式 ϕ；

(3) 所有机器人 $r \in \mathcal{N}$ 完成自身执行策略 τ_r 的总时间损耗 T^{colla} 最小。

2.5 本章小结

本章首先介绍了机器人任务规划中的线性时序逻辑理论的基本概念，包括满足线性时序逻辑表达式的命题序列、时序逻辑表达式对应的自动机模型等，用于描述工作环境中的任务要求以及其中的时序约束。然后根据机器人的区域转移能力和可提供的技术功能，对工作环境进行了离散化建模。根据现实问题场景中的任务要求特点，我们假设工作环境中存在个体任务和协同任务两类任务，定义了这两种任务的满足条件以及相互之间的区别。最后，我们形式化描述了本文研究的问题，即给定多机器人系统的全局协同任务 LTL_f 表达式和系统中每个机器人个体任务 LTL_f 表达式的情况下，如何高效求解机器人的执行策略，使得个体和协同任务表达式都能被满足，同时优化机器人完成所有任务的总时间损耗。

中国知网 https://www.cnki.net

3 时序约束下的协同任务分解与分配

时序约束下的经典多机器人协同任务规划算法通常依赖于构建一个全局的自动机模型，该模型的构造过程包括：（1）将时序任务 LTL_f 表达式转换为对应的自动机模型;（2）将工作环境中所有机器人的加权切换系统进行融合，构造联合加权切换系统;（3）将上述自动机模型和联合加权切换系统进行融合，构建一个全局自动机，然后利用图搜索算法在自动机上搜索执行策略。由于上述构造过程中考虑的状态组合数随着机器人数量的增长而出现指数爆炸，这限制了经典算法可以适用的问题规模[31]。

本章考虑对协同任务 LTL_f 表达式进行独立性分解，并在协同任务要求和时序逻辑的约束下，将协同任务分配给个体机器人。这将有助于在后续任务规划过程中，机器人通过本地规划求解执行策略，从而分摊计算量，降低任务规划问题的规模并提升求解效率。本章首先利用文献 [34] 中提出的独立性命题序列识别方法，通过在协同任务 LTL_f 表达式对应的自动机模型上判断命题序列（协同任务序列）之间的独立性，并对命题序列进行独立性分解。然后通过建立 SMT 模型来求解协同任务分配问题。

3.1 基于自动机的时序任务独立性分解

从第 2 章可知，将协同任务 LTL_f 表达式转化为对应的非确定性有限自动机（NFA）$\mathcal{F}$ 后，我们只需寻找 $\mathcal{F}$ 中的一条接收路径对应的命题序列，就可以满足该 LTL_f 表达式 ϕ。在实际场景中，各项协同任务之间可能存在时序约束，也可能是相互独立的，通过识别协同任务 LTL_f 表达式中能独立执行的部分，可以降低机器人在任务规划和实际执行中的耦合情况，从而降低规划问题的规模，并减少执行中的通信量。在本小节中，我们将基于协同任务 LTL_f 表达式对应的 NFA 模型，对其进行独立性分解，具体包括分解状态识别、状态转移关系筛除以及关键序列选择与分解三个步骤，后文将分别进行阐述。

3.1.1 分解状态识别

给定一个协同任务的命题序列 σ，若该序列在一个 NFA $\mathcal{F}$ 上描述了一条接收路径 ρ_F，则我们可以通过 ρ_F 中的“分解状态”将该命题序列 σ 分解为 $S(\geq 1)$ 个可以相互

中国知网 https://www.cnki.net

独立执行的子命题序列[34]，即 $\sigma = \sigma^1; \sigma^2; \cdots; \sigma^S$。分解状态的判断方法见定义 3.1。

定义 3.1 (分解状态). 假设 $q \in Q$ 是 NFA $\mathcal{F}$ 中的一个状态，且 $\sigma_1\sigma_2$ 是 $\mathcal{F}$ 中的一个关键序列，其中 σ_1 描述了 $\mathcal{F}$ 中从某个初始状态到状态 q 的一条路径，且 σ_2 描述了 $\mathcal{F}$ 中从状态 q 到 $\mathcal{F}$ 中某个接收状态的一条路径，则 q 为分解状态当且仅当 $\sigma_2\sigma_1$ 在 $\mathcal{F}$ 中描述了一个接收路径。

上述分解过程满足以下两个性质[34]：

- 独立性。$\forall\, k, j \in \mathcal{S}, k \neq j$，子命题序列 σ^k 的执行过程不会影响其它的子命题序列 σ^j 的执行，其中 $\mathcal{S} := [1 : S]$。
- 完整性。若 $\forall\, k \in \mathcal{S}$，$\sigma^k$ 都被执行，等价于原命题序列 σ 被执行。

对于协同任务 LTL_f 表达式 ϕ 对应的 NFA $\mathcal{F}$ 而言，描述 $\mathcal{F}$ 中的一个接收路径 ρ_F 的一个关键序列 σ 实际上是一个由协同任务的命题集合所组成的序列。对于子序列 σ^k 而言，它的第 m 个元素 $\sigma^k(m)$ 是一个由协同任务对应的原子命题组成的集合，定义为 $\sigma^k(m) := \{ct^k_{(m,j)}\}_{j\in[1:|\sigma^k(m)|]}$，其中 $ct^k_{(m,j)} \in \widetilde{\mathbb{T}}$ 表示 σ^k 中的第 m 个元素对应的集合中的第 j 个原子命题（或协同任务）。为便于理解，后文中若不加特殊说明，我们将协同任务 ct 和其对应的原子命题 π^{ct} 统一称为协同任务 ct。需要注意的是，描述接收路径 ρ_F 的关键序列 σ 中的每一个元素对应的协同任务原子命题集合，可能包括正命题和非命题，这是因为 $\mathcal{F}$ 中的状态转移条件，既包括满足该状态转移条件要求哪些命题为真，也要求了哪些命题为非。在本文中，我们只考虑关键序列 σ 中的真命题，并假设非命题由机器人在实际执行中通过通信来避免。例如，要满足转移条件 $\pi^{ct_1} \wedge (\neg\pi^{ct_2})$，则需要在执行协同任务 ct_1 的同时，避免协同任务 ct_2 被执行。而在规划阶段，我们仅考虑 ct_1，并假设“避免 ct_2 同时被执行”这一限制条件会由机器人在实际执行中通过通信来保证。将非命题的检查放到实际执行阶段，可以有效降低任务规划阶段的复杂度，否则我们就需要在任务规划时考虑其它机器人所有可能的状态组合。同样的假设也可以在文献 [61] 中找到。

根据上述分解状态的两个性质，子序列 σ^k 的执行是相互独立的。此外，根据 σ 在自动机 $\mathcal{F}$ 上的接收条件，同一个 σ^k 内部的协同任务的执行必须按照其在 σ^k 中的顺序。具体而言，σ 中每一项协同任务的执行需要满足以下两种约束：

(1) 同步约束。$\forall\, ct^k_{(m,j)}$，$ct^k_{(m,j')} \in \sigma^k(m)$，$j \neq j'$，$ct^k_{(m,j)}$ 和 $ct^k_{(m,j')}$ 需要同时执行；

sigma^k(m)对应多个协同任务，这些协同任务需要被同时执行

中国知网 https://www.cnki.net

(2) 次序约束。$\forall\, ct^k_{(m,j)}$，$ct^k_{(m',j')} \in \sigma^k, m < m'$，$ct^k_{(m',j')}$ 必须在 $ct^k_{(m,j)}$ 完成之后执行。

sigma^k中前后两个协同任务

经过分解状态识别与协同任务的命题序列独立性分解之后，上述两种约束对序列 σ 中协同任务之间的约束进行了一定程度的松弛，这可以在实际执行中减少机器人之间的通信，并有助于降低总时间损耗，见注 4.1。图 3.1中展示了图 2.2中 LTL$_f$ 表达式 ϕ 对应的 NFA，其中红色节点即为按照定义 3.1计算出的分解状态，经过该状态的接收路径被分解可以得到独立执行的子任务序列。

什么意思？怎么松弛了？独立性分解可以减少通信可以理解，然而两种约束的松弛指的是？

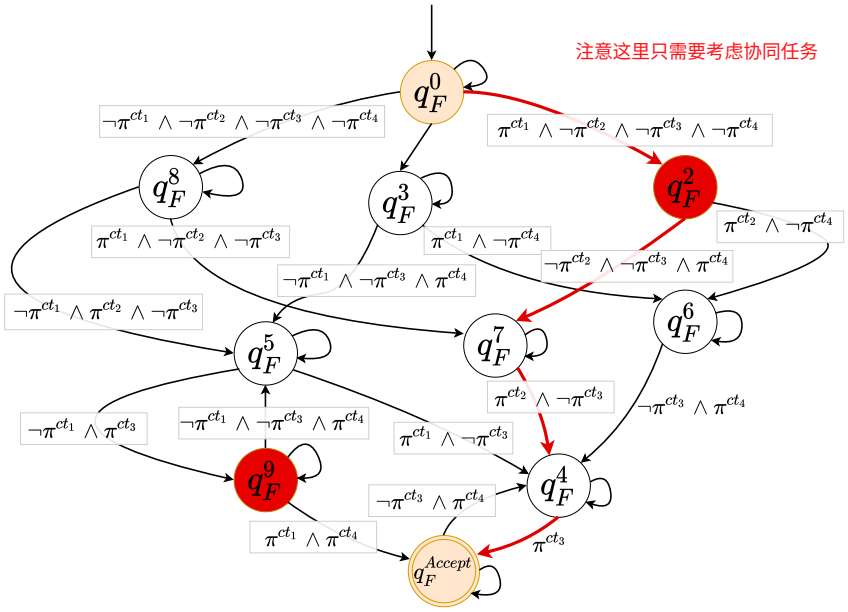


图 3.1: 图 2.1对应的状态缩减后的 NFA 模型以及其中的分解状态

3.1.2 状态转移关系筛除

为避免 NFA 模型中选取的任务序列无法被当前机器人群体所满足，在此我们需要移除 $\mathcal{F}$ 中的此类状态转移关系。$\mathcal{F}$ 中的状态转移条件被表示为 CNF 形式（见第 2.2 节），其中存在两种情况，会使得状态转移条件无法满足：

- 若自动机中的某条边上的**所有**用逻辑或（OR，$\vee$）连接的转移条件无法被当前的多机器人群体所满足，包括数量要求和功能要求，则将这条边移除；
- 若自动机上的某条边上的**某项**用逻辑与（AND，$\wedge$）连接的转移条件无法被当前的机器人群体所满足，则将该项转移条件移除。

中国知网 https://www.cnki.net

完成上述状态转移关系筛除后，$\mathcal{F}$ 中所有的状态转移关系对于当前的多机器人群体而言，均为可行的状态转移。为了符号简洁起见，除非特别说明，后文中默认 $\mathcal{F}$ 为进行状态转移关系筛除后的 $\mathcal{F}$。

图 3.1 展示了图 2.1 对应的状态缩减后的 NFA 模型。在图 2.2 所示的仿真案例中，我们假设每一项协同任务都需要至少 2 个机器人配合完成，因此图 3.1 中所有要求超过一个命题为真的状态转移条件都被移除，如原始的 NFA 模型中的状态转移条件 $\pi^{ct_1} \wedge \pi^{ct_2} \wedge \pi^{ct_3} \wedge \pi^{ct_4}$ 无法被 3 个机器人满足，因此在图 3.1中被移除。

3.1.3 关键序列选择与分解

为了满足协同任务 LTL_f 表达式 ϕ，我们需要在 NFA $\mathcal{F}$ 中搜索一条接收路径，并提取出其对应的关键序列 σ。机器人执行该关键序列 σ 即可保证满足协同任务 LTL_f 表达式 ϕ。选择不同的协同任务命题序列 σ 将影响到任务完成的效率，而遍历所有满足条件的命题序列 σ 是十分低效的。直观来看，如果我们可以选择出在状态转移时对机器人的要求较低，包括了尽可能多的分解状态，以及尽可能短的协同任务命题序列，将有助于提高协同任务的执行效率。为了降低算法的整体复杂度，在本文中我们直接选择 NFA $\mathcal{F}$ 中的一条最短路径 ρ_F 对应的关键序列 σ 作为选定的协同任务命题序列。之后，我们利用第 3.1.1 节中的分解方法将协同任务序列 σ 分解为可以独立执行的子序列。在第 5 章的实验中，我们验证了在上述选定的协同任务序列下，机器人的执行策略具有良好的性能表现。例如，在图 3.1中，最终选择的接收路径（图中标为红色）为 $\rho_F = q_F^0\, q_F^2\, q_F^7\, q_F^4\, q_F^{Accept}$，其对应的关键序列为 $\sigma = \{\pi^{ct_1}\}\{\pi^{ct_4}\}\{\pi^{ct_2}\}\{\pi^{ct_3}\}$，可以被分解状态 q_F^2 划分为 $\sigma^1 = \{\pi^{ct_1}\}$ 和 $\sigma^2 = \{\pi^{ct_4}\}\{\pi^{ct_2}\}\{\pi^{ct_3}\}$ 两个独立子序列。

3.2 基于 SMT 模型的协同任务分配

本节考虑协同任务的分配问题。将协同任务分配给个体机器人，同时需要考虑满足协同任务对机器人数量和类型的要求，以及协同任务之间的时序约束。由于时序约束的存在，不合理的任务分配结果可能会导致机器人在执行各自的执行策略时发生死锁等状况，从而导致任务失败。在本节中，我们通过建立一个 SMT 模型来描述协同任务的各项约束，包括：协同任务对机器人数量和类型的要求；机器人的物理限制；以及在通信

中国知网 https://www.cnki.net

受限情况下保证协同任务执行次序的约束等。通过利用 SMT 求解器，我们可以快速获得可行的协同任务分配方案，并用于后续机器人的任务规划和协同。下文将首先介绍协同任务分配问题的建模过程，并进而提出基于 SMT 求解特点的任务分配方案的迭代求解与快速筛除机制。

3.2.1 协同任务分配问题建模

在获得分解后的协同任务序列 $\sigma = \sigma^1; \sigma^2; \ldots; \sigma^S$ 以及对应的 NFA $\mathcal{F}$ 中的接收路径 ρ_F 后，本节我们解决协同任务的分配问题，以满足每项协同任务对机器人数量和类型的要求，并考虑在任务区域之间通信受限的情况下如何确保协同任务的执行时序。

为直观起见，我们给 σ^k 中的所有任务 $ct^k_{(m,j)}$ 一个新的下标 l，定义为 $ct^k_l := ct^k_{(m,j)}$，其中 $l = \sum_{i=1}^{m-1} |\sigma^k(i)| + j$，$\forall\, m \in \left[1 : |\sigma^k|\right]$，$k \in \mathcal{S}$。$\mathcal{S}$ 为独立子命题序列的下标集合。这样，σ^k 中的第 l 个任务可以表示为 ct^k_l。随后，对每一个机器人 r，我们定义一个布尔变量的集合 $\mathcal{X}_i := \{x_r^{(k,l)} | ct^k_l \in \sigma^k, k \in \mathcal{S}\}$ 来指示任务分配的结果：变量 $x_r^{(k,l)}$ 为真当且仅当机器人 r 被分配到执行协同任务 ct^k_l。

下面我们将构建一个 SMT 模型来建模协同任务分配问题，包括以下三种约束：

（1）协同约束。可行的协同任务分配方案需要满足每一项协同任务所要求的机器人类型和数量，即：$\forall\, \sigma^k, \forall\, ct^k_l \in \sigma^k$,

$$\sum_{r \in \mathcal{N}} \mathbf{1}\left(x_r^{(k,l)} \bigwedge r \in \mathcal{N}_j\right) \geq m_j^{ct}, \forall j \in I_{ct}$$

为真，其中 $\mathbf{1}(\cdot)$ 为符号函数，定义为 $\mathbf{1}(\text{true}) = 1$，$\mathbf{1}(\text{false}) = 0$。

（2）物理约束。同一个机器人无法同时参与到两项协同任务中，即 $\forall\, \sigma^k, \forall\, ct^k_{l_1}, ct^k_{l_2} \in \sigma^k(m), l_1 \neq l_2$,

$$\neg x_r^{(k,l_1)} \bigvee \neg x_r^{(k,l_2)}, \forall\, r \in \mathcal{N}$$

为真。

（3）通信约束。对于两项需要连续执行的协同任务而言，它们分别被分配到的两个机器人集合的交集非空。即 $\forall\, \sigma^k$,

$$\bigvee_{r \in \mathcal{N}} \left(\bigvee_{l \in \{l^k_{m-1}+1, \ldots, l^k_m\}} x_r^{(k,l)} \bigwedge \bigvee_{l \in \{l^k_m+1, \ldots, l^k_{m+1}\}} x_r^{(k,l)} \right)$$

中国知网 https://www.cnki.net

为真，其中 $l_m^k = \sum_{j=1}^{m} |\sigma^k(j)|$，$\forall\, m \in [1 : |\sigma^k| - 1]$。

需要注意的是，上述约束（3）为可选约束。然而，当工作环境中的任务区域之间的通信链路受限时，在 SMT 模型中施加约束（3）可以保证每个机器人 $r \in \mathcal{N}$ 在执行被分配到的协同任务 ct 时，只需要与机器人 $j \in \mathcal{R}(ct)\backslash\{r\}$ 通信。这里函数 $\mathcal{R} : ct_l^k \to 2^{\mathcal{N}}$ 将协同任务 ct_l^k 映射到其分配到的机器人集合。实际上，由于约束（3）中要求 $\forall\, m \in \big[1 : |\sigma^k| - 1\big]$，$\mathcal{R}(\sigma^k(m)) \bigcap \mathcal{R}(\sigma^k(m+1)) \neq \emptyset$，机器人 $r \in \mathcal{R}(\sigma^k(m)) \bigcap \mathcal{R}(\sigma^k(m+1))$ 将在实际执行中保证 $\sigma^k(m)$ 和 $\sigma^k(m+1)$ 中协同任务的执行顺序，因此无需额外的机器人之间的通信。根据 SMT 约束（2）和第 3.1.1 节的同步约束（约束（2）），我们重载函数 $\mathcal{R}$ 为 $\mathcal{R}(\sigma^k(m)) := \bigcup_{ct \in \sigma^k(m)} \mathcal{R}(ct)$。

注 3.1. 通信约束（3）可以被选择性地施加在需要连续执行的协同任务之间，例如由于某些连续执行的协同任务所在的区域可能相距太远，而使机器人无法建立稳定的通信链路。通过施加约束（3）可以在这种情况下依然保证该协同任务之间的执行顺序。

最终，协同任务分配的 SMT 模型对应的逻辑表达式 f 即为上述约束（1）-（3）的与（$\wedge$）。在此，我们将满足 SMT 表达式 f 的一个可行解表示为 $X = \bigwedge_{r \in \mathcal{N}} \left(\wedge_{x_r^{(k,l)} \in \mathcal{X}_i} x_r^{(k,l)} \right)$，即 X 为集合 $\{\mathcal{X}_r\}_{r \in \mathcal{N}}$ 中所有布尔变量的与。

3.2.2 任务分配方案的迭代求解与快速筛除

通过利用 SMT 求解器迭代求解 SMT 表达式 f，我们可以遍历满足 SMT 表达式 f 的所有可行解。具体而言，在求得一个可行解 X 之后，我们在下一次求解前将 SMT 表达式更新为 $f = f \wedge \neg X$，其中 $\neg X$ 将作为一项约束避免求解到与 X 一样的解。根据可行解 X 中每个布尔变量的含义，我们可以方便地获得对应的协同任务分配方案。在一次求解完成后，布尔变量集合 $\{\mathcal{X}_r\}_{r \in \mathcal{N}}$ 将会被发送到每个机器人，以进行后续的任务规划过程并进而评估当前协同任务分配方案下机器人执行策略的性能。上述求解与评估过程将迭代进行，在允许的时间限制内，该迭代求解过程将持续优化当前的协同任务分配方案。

上述迭代求解过程，需要将每个 SMT 表达式 f 的可行解都发送给机器人以进行后续的性能评估，这在实际操作中会使算法变得低效。在此，我们提出一种可行解快速筛除机制，利用可行解的实际物理含义来筛除非最优的协同任务分配方案，从而避免对这些任务分配方案进行性能评估时导致算法变得低效的问题。

中国知网 https://www.cnki.net

任务分配方案快速筛除机制：若 $\exists m < n$，$m, n \in \mathbb{N}^+$，使得 $\forall r \in \mathcal{N}, \mathcal{X}_r^n \geq \mathcal{X}_r^m$，则 f 可以直接更新为 $f \wedge \neg X^n$，而无需进行后续的性能评估过程，其中 $\mathcal{X}_r^m$，$\mathcal{X}_r^n$ 分别是机器人 r 在第 m 次和第 n 次迭代求解中得到的可行的协同任务分配方案。此处 $\mathcal{X}_r^n \geq \mathcal{X}_r^m$ 表示 $\forall {}^n x_r^{(k,l)} \in \mathcal{X}_r^n$，${}^m x_r^{(k,l)} \in \mathcal{X}_r^m$ 而言，满足 $\mathbf{1}({}^n x_r^{(k,l)}) \geq \mathbf{1}({}^m x_r^{(k,l)})$。

在上述快速筛除机制中，中心服务器需要维护一个任务分配方案集合（实际上是可行解 X 的集合），该集合中包含没有被筛除的协同任务分配方案。当 SMT 求解器计算出一个新的协同任务分配方案时，该方案 X 将按照上述机制，与任务分配方案集合中已有的元素依次进行比较：若满足筛除条件，则 X 将被筛除；否则若与集合中的所有元素比较后都不符合筛除条件，则将该协同任务分配方案发送给机器人，以进行后续的任务规划，同时将 X 加入到中心服务器维护的协同任务分配方案集合中。关于该任务分配方案快速筛除机制的筛除效果见第 5 章的实验结果。

3.3 本章小结

本章基于多机器人的协同任务 LTL_f 表达式，对其进行了独立性分解，并将协同任务分配给多机器人。首先，将协同任务 LTL_f 表达式转化为对应的 NFA 模型，根据实际物理意义对 NFA 模型的状态转移关系进行筛除后，在 NFA 模型上提取出可以满足协同任务 LTL_f 表达式的协同任务序列，并利用分解状态的概念对该协同任务序列进行分解，以降低实际任务执行时机器人之间的耦合。

进一步，本章通过建立并求解 SMT 模型，将协同任务序列中的协同任务按照其满足条件、物理约束和通信限制等约束分配给多机器人，以保证多机器人可以满足协同任务 LTL_f 表达式的要求。最后，为了搜索最佳的协同任务分配方案，本章利用 SMT 的求解特性，通过将当前可行解取反后加入原 SMT 表达式，实现对 SMT 模型的迭代求解以遍历所有可行的协同任务分配方案。为了提高上述迭代遍历过程的效率，本章提出了一种任务分配方案快速筛除算法，利用 SMT 可行解的实际物理含义快速筛除非最优的协同任务分配方案，从而避免了将非最优的协同任务分配方案传递给机器人，提高了任务分配方案的优化效率。

中国知网 https://www.cnki.net

4 多机器人执行策略的求解与优化

上一章对全局协同任务要求进行了独立性分解与协同任务的分配，该过程使得我们可以将机器人的任务规划过程分摊到每个机器人本地进行。每个机器人不仅需要满足个体任务 LTL_f 表达式的要求，还需要满足被分配到的协同任务。在本章中，我们将对上述机器人任务规划问题进行求解。

进一步，我们发现通过本地求解得到的机器人执行策略，在执行协同任务时的效率较低。这是由于机器人在进行本地规划时，仅仅从自身时间最优的角度来规划参与到协同任务的时间，而没有考虑其它机器人的参与时间，因此由于机器人到达协同任务区域的时间不同而会产生低效的协同等待时间。本章将进一步针对机器人执行协同任务时的时间损耗进行执行策略的调整优化。

下面将分别从多机器人执行策略的求解与优化两个部分进行详细阐述。

4.1 多机器人执行策略求解与执行

4.1.1 多机器人执行策略求解

求解机器人的执行策略的核心问题是如何保证同时满足每个机器人 $r \in \mathcal{N}$ 的个体任务 LTL_f 表达式 φ_r，以及机器人被分配到的协同任务与其相互间的时序约束（如第 3.1.1 章所述）。

本小节中，我们将通过构建一个更新后的机器人个体任务 LTL_f 表达式，来融合上述两种任务的时序要求。另外，将个体任务要求和协同任务要求进行融合，也有利于机器人统筹规划自身的执行策略，以优化整体的时间效率。在构建完上述机器人个体任务 LTL_f 表达式后，我们可以利用经典的时序逻辑任务规划算法，在本地求解一个单机器人任务规划问题，从而获得可以同时满足机器人的个体任务 LTL_f 表达式以及被分配到的协同任务的执行策略。在上面的求解过程中，计算量将被分摊到每个机器人本地，因此可以有效提高该算法在大规模机器人任务规划中的求解效率。以下介绍具体步骤。

给定一个可行的协同任务分配方案（SMT 模型的可行解）$\mathcal{X}$，构建集合 $\widetilde{\mathbb{T}}_r^k := \{ct_l^k \mid x_r^{(k,l)} \text{ 为真}\}$，$\forall\, k \in \mathcal{S}, r \in \mathcal{N}$。令 $\widetilde{\mathbb{T}}_r := \bigcup_{k\in\mathcal{S}} \widetilde{\mathbb{T}}_r^k$，其中，$\widetilde{\mathbb{T}}_r$ 中的所有协同任

中国知网 https://www.cnki.net

务将按照下标 k 和 l 递增的顺序排列。

进而，对机器人 r 而言，更新后的个体任务 LTL$_f$ 表达式 $\widetilde{\varphi}_r$ 定义为 $\widetilde{\varphi}_r = \varphi_r \bigwedge \phi_r$。其中，$\phi_r$ 是描述了集合 $\widetilde{\mathbb{T}}_r$ 中的协同任务及其之间的时序约束的 LTL$_f$ 表达式。构建 ϕ_r 和 $\widetilde{\varphi}_r$ 的过程包含两个步骤：

步骤 1： 建模协同任务子命题序列内部的次序约束。$\forall\, k \in \mathcal{S}$，构建 $\widetilde{\mathbb{T}}_r^k$ 对应的协同任务表达式 ϕ_r^k。首先，定义 $\phi_r^k = \Diamond(ct_{l_1}^k)$，其中 $ct_{l_1}^k$ 是集合 $\widetilde{\mathbb{T}}_r^k$（按照下标 l 递增的顺序排序）的第一项协同任务。随后，使用 $ct_{l_m}^k \bigwedge \Diamond(ct_{l_{m+1}}^k)$ 来迭代地替换 ϕ_r^k 中的 $ct_{l_m}^k$，其中 $ct_{l_m}^k$ 和 $ct_{l_{m+1}}^k$ 是 $\widetilde{\mathbb{T}}_r^k$ 中两项连续的协同任务，且满足 $l_m < l_{m+1}$。

由第 3.1.1 节中所述，每个协同任务子命题序列可以独立执行而不会相互影响。然而，由于同一个子命题序列中的协同任务可能分配给多个机器人，而这些机器人执行上述独立子序列的次序有可能不同，从而造成死锁。为避免这种情况，我们将在步骤 2 中固定协同任务子命题序列的执行顺序。

步骤 2： 固定协同任务子命题序列之间的执行次序。对于每个机器人 r，我们将步骤 1 中获得的 $\{\phi_r^k\}_{k\in\mathcal{S}}$ 按照下标 k 增长的顺序进行排列。从 $k=1$ 到 $S-1$，迭代地将 $ct_{l_{max}}^k$ 替换为 $ct_{l_{max}}^k \bigwedge \Diamond(\phi_r^{k+1})$，其中 $ct_{l_{max}}^k$ 是排序后的集合 $\widetilde{\mathbb{T}}_r^k$ 中的最后一项协同任务。最终，$\phi_r = \phi_r^1$，且 $\widetilde{\varphi}_r = \varphi_r \bigwedge \phi_r$。

需要指出的是，上述步骤 2 给相互独立的协同任务子序列的执行施加了次序约束，这看似削弱了在第 3.1 节中进行的独立性分解的意义。然而，我们将进一步说明独立性分解步骤在机器人的实际部署中起到了降低通信量的作用，见注 4.1。

注 4.1. 在 NFA $\mathcal{F}$ 中对关键序列 σ 进行独立性分解可以降低实际部署中的通信量。例如，假设存在一个关键序列 $\sigma = \{ct_1^{k_1}\}\{ct_1^{k_2}\}$，其中 $\sigma^{k_1} = \{ct_1^{k_1}\}$ 和 $\sigma^{k_2} = \{ct_1^{k_2}\}$ 是两个独立的协同任务子序列。若 $\mathcal{R}(ct_1^{k_1}) \cap \mathcal{R}(ct_1^{k_2}) = \emptyset$，则这两项协同任务 $ct_1^{k_1}$ 和 $ct_1^{k_2}$ 可以相互独立地执行。相反，若不判别上述两个协同任务子序列的独立性，则所有属于集合 $\mathcal{R}(ct_1^{k_2})$ 的机器人都需要等待协同任务 $ct_1^{k_1}$ 的完成，从而增加了通信量以及总体时间消耗。

完成对更新后的个体任务 LTL$_f$ 表达式 $\widetilde{\varphi}_r$ 的构造后，每个机器人 $r \in \mathcal{N}$ 可以利用转化工具 [62-63] 获得 $\widetilde{\varphi}_r$ 对应的 NFA $\mathcal{F}_r$。通过将加权切换系统 wTS_r 与 $\mathcal{F}_r$ 相结合，每个机器人 r 可在本地构造一个乘积自动机（Product Automaton）$\mathcal{P}_r$，并从中搜索得到自身的执行策略。上述乘积自动机 $\mathcal{P}_r$ 的构造过程见定义4.1。个体机器人的乘积自动机示意图见图 4.1。

中国知网 https://www.cnki.net

定义 4.1 (乘积自动机)**.** 加权切换系统 wTS_r 和 NFA $\mathcal{F}_r$ 的乘积自动机表示为一个元组 $\mathcal{P}_r = wTS_r \otimes \mathcal{F}_r = \langle \mathcal{Q}_P, \mathcal{Q}_P^0, \rightarrow_P, \omega_P, \mathcal{Q}_P^F \rangle$，其中，

- $\mathcal{Q}_P = \mathcal{Q}_r \times \mathcal{Q}_F$；
- $\mathcal{Q}_P^0 = q_i^0 \times \mathcal{Q}_F^0$；
- $\rightarrow_P \subseteq \mathcal{Q}_P \times \mathcal{Q}_P$，并满足 $\forall\, (q_P, q_P') \in \rightarrow_P$，有 $(\Pi_r q_P, \Pi_r q_P') \in \rightarrow_i$ 和 $L_r(\Pi_r q_P) \vDash \delta(\Pi_F q_P, \Pi_F q_P')$ 成立。这里 Π_r 和 Π_F 分别表示在 wTS_r 和 $\mathcal{F}_r$ 状态空间上的投影；
- $\omega_P(q_P, q_P') = \omega_i(\Pi_r q_P, \Pi_r q_P')$；
- $\mathcal{Q}_P^F \subseteq \mathcal{Q}_r \times \mathcal{Q}_F^F$。

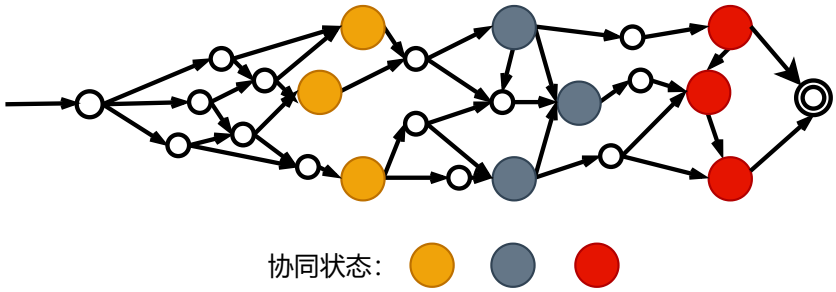


图 4.1: 个体乘积自动机示意图

利用 Dijkstra 算法[64]，每个机器人 $r \in \mathcal{N}$ 可以在乘积自动机 $\mathcal{P}_r$ 上搜索一条最短的接收路径 ρ_r。当每个机器人 $r \in \mathcal{N}$ 执行从路径 ρ_r 中提取出的关键序列 σ 时，即可满足机器人 r 的个体任务 LTL_f 表达式 φ_r；同时，所有机器人的联合行动可以满足全局协同任务 LTL_f 表达式 ϕ。上述任务执行过程也可以保证遵守机器人在工作环境中的移动约束。机器人 r 的执行策略（即在任务区域之间的移动方案）τ_r 可以通过将 ρ_r 映射到 wTS_r 的状态空间中获得，即 $\tau_r = \Pi_r \rho_r$。

4.1.2 协同状态识别与多机器人任务执行机制

为了明确获知机器人在哪些任务区域需要执行一定的动作，哪些区域只是临时通过，我们需要识别出乘积自动机 $\mathcal{P}_r$ 中对应于执行协同任务 $ct \in \widetilde{\mathbb{T}}_r$ 的协同状态，具体见定义4.2。

定义 4.2 (协同状态)**.** $\forall\, q_P \in \mathcal{Q}_P$, $q_P \in \mathcal{C}(ct)$ 当且仅当其满足以下两个条件。其中，$\mathcal{C}(ct)$ 是协同任务 ct 在 $\mathcal{P}_r$ 中对应的所有协同状态的集合。

中国知网 https://www.cnki.net

- $ct \in L_r(\Pi_r q_P)$;
- $\exists q'_P \in \mathcal{Q}_P, q_P \in \delta(q'_P, ct)$，且 $\Pi_{\mathcal{F}} q'_P \neq \Pi_{\mathcal{F}} q_P$.

上述第二个条件保证了机器人会在状态（任务区域）q_P 处执行一定的动作，而非临时经过。

执行策略的实施：给定每个机器人 $r \in \mathcal{N}$ 的执行策略 τ_r，机器人 r 按照 τ_r 中的顺序依次访问这些状态对应的任务区域，并遵守以下规则：

(1) 若机器人 r 到达某个状态 $q_P \in \mathcal{Q}_P$，且该状态满足 $L_r(q_P) \subseteq \overline{\mathbb{T}}_r$，则该机器人需要独自执行集合 $L_r(q_P)$ 中的所有任务（使该集合中的所有任务对应的原子命题为真），随后转移到下一个状态；

(2) 若机器人 r 到达某个状态 $q_P \in \mathcal{Q}_P$，且该状态满足 $\exists\, ct \in \widetilde{\mathbb{T}}_r, q_P \in \mathcal{C}(ct)$，则机器人 r 需要在该区域等待集合 $\mathcal{R}(ct)\backslash\{r\}$ 中的所有机器人到达，然后一起执行该区域内的协同任务。

一旦某个机器人 r 移动到 τ_r 的最后一个状态对应的任务区域，并完成了该区域内的任务，则该机器人 r 的任务完成。

命题 4.1 (完备性)**.** 在假设 2.1下，如果问题 1 存在可行解，则上述方法可以获得多机器人执行策略 $\{\tau_r\}_{r\in\mathcal{N}}$，使得当机器人异步执行自身的执行策略并在执行协同任务时遵守与其它机器人的同步约束，则可以满足每个机器人 $r \in \mathcal{N}$ 的个体 LTL_f 表达式 φ_r 和全局协同 LTL_f 表达式 ϕ。

证明. 若存在问题 1 的解 $\{\tau_r\}_{r\in\mathcal{N}}$，则该解满足：（1）机器人执行协同任务的动作所产生的命题序列，可以在协同任务 LTL_f 表达式对应的 NFA $\mathcal{F}$ 中（第 3.1.2 章）描述一条接收路径；（2）每个机器人 r 在执行个体任务时所产生的命题序列，可以在 $\widetilde{\varphi}_r$ 对应的 $\mathcal{F}_r$ 中描述一条接收路径 ρ_r，因为 $\mathcal{F}_r$ 包含了所有可以满足任务表达式 φ_r 的命题序列，以及分配给机器人 r 的协同任务之间的时序约束。显然，所有的可行解（机器人执行策略）都会出现在本文所述的方法的搜索空间中。

我们证明机器人在实施执行策略时，将会在每个机器人的个体 LTL_f 表达式对应的 NFA 中描述一条接收路径，且机器人的联合动作将会在全局协同任务 LTL_f 表达式对应的 NFA 中描述一条接收路径。首先，在我们提出的执行策略实施方式中，机器人异步

中国知网 https://www.cnki.net

执行自身的执行策略，并在执行协同任务时与其它机器人进行同步协同。与之相对，经典时序逻辑任务规划算法中假设机器人在执行任务时是严格同步的。根据定义 2.3，机器人仅需在执行协同任务（使协同任务对应的命题为真）时才需要与其它机器人同步。当机器人在区域之间转移而没有执行具体任务时，没有原子命题为真，这种情况也是满足自动机转移条件的，因为在 $\mathcal{F}_r$ 中的每个状态处总存在一个自循环，且其对应的转移条件始终为真，这是由于我们在 finite LTL 的语法中移除了 next 运算符。

其次，在机器人执行自身的执行策略时，生成的任务序列（为真的原子命题序列）中协同任务和个体任务可能是交叉出现的。在这种情况下，个体任务 LTL_f 表达式和全局协同任务 LTL_f 表达式也会被满足。因为我们假设了个体任务和协同任务的执行是相互独立的（见假设 2.1），所以机器人执行个体任务时，协同任务 LTL_f 表达式对应的 NFA 上的状态就停留在当前状态的自循环上（始终为真）；反之机器人执行协同任务时，个体任务对应的自动机上的状态也停留在原来状态的自循环上。

综上所述，若存在问题 1 的解，则本文中提出的上述方法可以找到这个解。并且通过本文提出的执行策略实施方式，可以保证既满足机器人个体任务 LTL_f 表达式的要求，也可以满足全局协同任务 LTL_f 表达式的要求。 □

根据上述机器人执行策略的实施方式，当机器人执行某项协同任务时，由于需要与其它机器人进行同步协同，可能导致机器人在协同任务区域出现协同等待时间。这是由于机器人到达该协同任务区域的时间不同而导致的，见图 4.2 所示。出现这一情况是因为机器人在本地计算自身的执行策略时，仅从自身的角度选择了最优的方案，而并未考虑与其它机器人的协同。下一节中，我们将对机器人执行协同任务时的效率进行优化。

4.2 基于混合整数规划的集中式执行策略优化

在上一节中，每个机器人 $r \in \mathcal{N}$ 通过本地构建个体乘积自动机，求得了自身的执行策略 τ_r，且该执行策略被实施时可以保证分别满足机器人个体任务 LTL_f 表达式 φ_r 和全局协同任务 LTL_f 表达式 ϕ。但上述本地求解得到的执行策略，在实际执行时可能出现由于机器人到达协同任务区域时间不同而导致的低效等待问题，使多机器人完成任务的时间效率降低。本小节中，我们提出一种基于混合整数线性规划（Mixed Integer Linear Programming, MILP）建模的优化模型，来求解多机器人的最优执行策略。首先，

中国知网 https://www.cnki.net

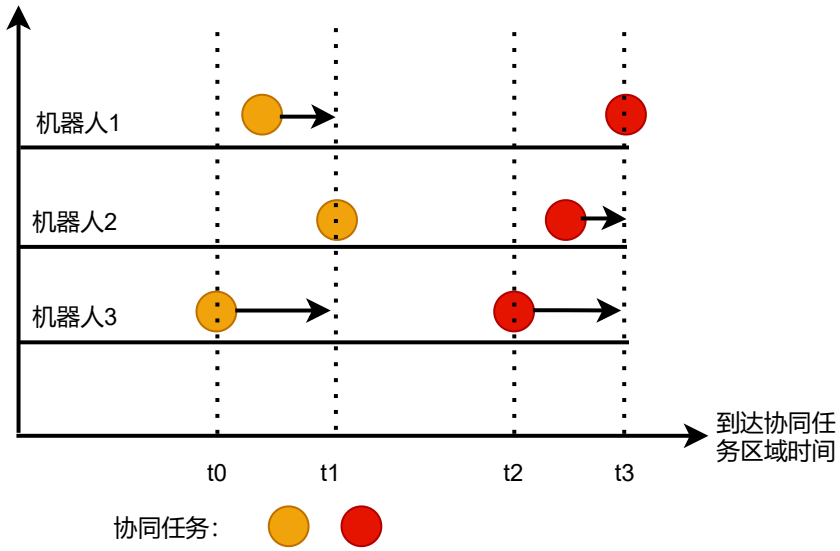


图 4.2: 协同等待时间示意图

我们对机器人的个体乘积自动机进行状态缩减，仅保留与执行协同任务相关的状态；随后，基于状态缩减后的个体乘积自动机，我们建立 MILP 模型；最后利用求解器求解上述 MILP 优化模型，获得在给定协同任务分配方案下的多机器人最优执行策略。

4.2.1 个体乘积自动机状态缩减

为了优化执行策略的实际执行效率，我们仅关注与执行协同任务相关的状态。原因在于，机器人执行所有任务消耗的总时间取决于：（1）每个机器人在不考虑协同等待时间的情况下的任务执行时间；（2）在执行协同任务时所消耗的额外的协同等待时间。因此，在保留每个机器人的个体乘积自动机中的初始状态和接收状态的情况下，只保留执行协同任务相关的状态，因素（1）不会受到影响，并有利于在考虑因素（2）时减小问题的规模。

我们将进行状态缩减后的机器人 r 的个体乘积自动机 $\mathcal{P}_r$ 表示为 $\overline{\mathcal{P}}_r$。$\overline{\mathcal{P}}_r$ 中只保留三种状态：（1）所有的初始状态 $q_P^0 \in \mathcal{Q}_P^0$；（2）所有接收状态 $q_P^F \in \mathcal{Q}_P^F$；（3）所有协同状态 $q \in \mathcal{C}(ct)$，$\forall\, ct \in \widetilde{\mathbb{T}}_r$。状态缩减后的个体乘积自动机的构造过程见定义4.3。

定义 4.3. 进行状态缩减后的机器人 r 的个体乘积自动机表示为一个元组 $\overline{\mathcal{P}}_r = \langle \mathcal{Q}_{\overline{P}}, \mathcal{Q}_{\overline{P}}^0, \rightarrow_{\overline{P}}, \omega_{\overline{P}}, \mathcal{Q}_{\overline{P}}^F \rangle$，其中：

- $\mathcal{Q}_{\overline{P}} \in \left(\bigcup_{ct \in \widetilde{\mathbb{T}}_r} \{\mathcal{C}(ct)\}\right) \bigcup \mathcal{Q}_{\overline{P}}^0 \bigcup \mathcal{Q}_{\overline{P}}^F$；
- $\mathcal{Q}_{\overline{P}}^0 = \mathcal{Q}_P^0$；

中国知网 https://www.cnki.net

- $\rightarrow_{\overline{P}} \subseteq \mathcal{Q}_{\overline{P}} \times \mathcal{Q}_{\overline{P}}$。其中，$\forall\,(q_{\overline{P}}, q'_{\overline{P}}) \in \rightarrow_{\overline{P}}$, 在 $\mathcal{P}_i$ 中存在一条从 $q_{\overline{P}}$ 到 $q'_{\overline{P}}$ 的路径 ρ。$\rightarrow_{\overline{P}}$ 中总共有三种类型的边，满足 $\forall (q_{\overline{P}}, q'_{\overline{P}}) \in \rightarrow_{\overline{P}}$:
  1) 若 $q_{\overline{P}} \in \mathcal{C}(ct), q'_{\overline{P}} \in \mathcal{C}(ct')$，$ct, ct' \in \widetilde{\mathbb{T}}_r$，则 ct 是集合 $\widetilde{\mathbb{T}}_r$ 中 ct' 的上一项任务；
  2) 否则，若 $q_{\overline{P}} \in \mathcal{Q}^0_{\overline{P}}$ 且 $q'_{\overline{P}} \in \mathcal{C}(ct)$，则 ct 一定是集合 $\widetilde{\mathbb{T}}_r$ 中的第一项任务；
  3) 否则，若 $q_{\overline{P}} \in \mathcal{C}(ct)$ 且 $q'_{\overline{P}} \in \mathcal{Q}^F_{\overline{P}}$，则 ct 一定是集合 $\widetilde{\mathbb{T}}_r$ 中的最后一项任务。
- $\omega_{\overline{P}}(q_{\overline{P}}, q'_{\overline{P}}) = \sum_{k=1}^{|\rho|-1} \omega_P(\rho(k), \rho(k-1))$，其中 ρ 是 $\mathcal{P}_r$ 中状态 $q_{\overline{P}}$ 与状态 $q'_{\overline{P}}$ 之间的最短路径；
- $\mathcal{Q}^F_{\overline{P}} = \mathcal{Q}^F_P$。

直观来看，状态缩减后的乘积自动机 $\overline{\mathcal{P}}_r$ 具有分层式的结构：除第一层和最后一层外，每一层中都包含了集合 $\widetilde{\mathbb{T}}_r$ 中的某个协同任务的所有协同状态，且层与层之间的排布顺序满足对应的协同任务在 $\widetilde{\mathbb{T}}_r$ 中的顺序。图 4.3 展示了在仿真实验中，状态缩减后的个体乘积自动机的局部，从图中可以明显看出该自动机模型中状态的分层连接关系。

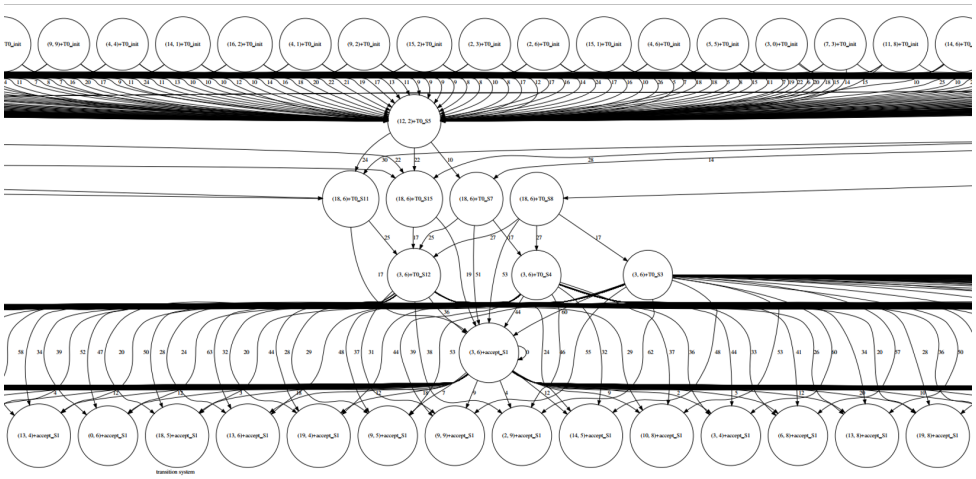


图 4.3: 状态缩减后的个体乘积自动机

命题 4.2. 对机器人 r 而言，状态缩减后的个体乘积自动机 $\overline{\mathcal{P}}_r$ 的构造时间复杂度为 $O(\Delta^2 \cdot |\widetilde{\mathbb{T}}_r| \cdot (E \cdot \lg E))$，其中 Δ 表示协同任务 $ct \in \widetilde{\mathbb{T}}_r$ 在 $\mathcal{P}_r$ 中对应的协同状态个数的规模，$E \cdot \lg E$ 表示利用 Dijkstra 算法在 $\overline{\mathcal{P}}_r$ 中搜索从状态 $q_{\overline{P}}$ 到状态 $q'_{\overline{P}}$ 的最短路径以构造定义4.3 中的边 $(q_{\overline{P}}, q'_{\overline{P}})$ 的复杂度，E 表示 $\mathcal{P}_r$ 的边数。

中国知网 https://www.cnki.net

4.2.2 基于混合整数规划模型的执行策略优化

本小节中，我们将基于上一小节中构造的状态缩减后的个体乘积自动机 $\overline{\mathcal{P}}_r$，建立 MILP 策略优化模型。由该 MILP 模型的解，我们可以得到每个机器人 $r \in \mathcal{N}$ 在 $\overline{\mathcal{P}}_r$ 上的一条接收路径，并通过将该路径映射回 $\mathcal{P}_r$ 得到机器人 r 的最优执行策略。MILP 模型可以通过利用如 Gurobi[65]等求解器进行求解。

首先，对于每个机器人 $r \in \mathcal{N}$，定义一个布尔变量集合 $\mathcal{Y}_r = \{y_r^{(i,j)} | (q_i, q_j) \in \rightarrow_{\overline{P}}\}$，其中，变量 $y_r^{(i,j)}$ 为真表示该变量对应的边 (q_i, q_j) 属于机器人 r 在 $\overline{\mathcal{P}}_r$ 上的最优接收路径。根据上述定义，可以通过求解后集合 $\mathcal{Y}_r$ 中变量的取值来获得机器人 r 的最优执行策略。通过施加以下约束，我们可以保证集合 $\mathcal{Y}_r$ 中的布尔变量取值能够在 $\overline{\mathcal{P}}_r$ 中对应一条接收路径，具体为：

$$y_r^{(i,j)} = 1 \ or \ 0, \forall \, (q_i, q_j) \in \rightarrow_{\overline{P}}, r \in \mathcal{N}; \tag{4-1}$$

$$\sum_{q_k \in Pre(i)} y_r^{(k,i)} = \sum_{q_j \in Sub(i)} y_r^{(i,j)} \leq 1, \forall \, q_i \in \mathcal{Q}_{\overline{P}} \backslash (Q_{\overline{P}}^0 \cap Q_{\overline{P}}^F); \tag{4-2}$$

$$\sum_{q_i \in \mathcal{Q}_{\overline{P}}^0} \sum_{q_j \in Sub(i)} y_r^{(i,j)} = 1; \tag{4-3}$$

$$\sum_{q_i \in \mathcal{Q}_{\overline{P}}^F} \sum_{q_k \in Pre(i)} y_r^{(k,i)} = 1. \tag{4-4}$$

这里 $Pre(i)$ 和 $Sub(i)$ 分别表示状态 q_i 在 $\overline{\mathcal{P}}_r$ 中的前继节点和后继节点的集合。

通过施加上述约束 $(1)-(4)$，集合 $\mathcal{Y}_r$ 中所有取值为真的布尔变量 $y_r^{(i,j)}$ 将会在机器人 r 的状态缩减后的个体乘积自动机 $\overline{\mathcal{P}}_r$ 中对应一条接收路径。

然后，我们对多机器人执行协同任务时的协同等待时间进行建模。对于每一项协同任务 $ct_l^k \in \widetilde{\mathbb{T}}_r$，我们分别定义两种辅助变量 $t_r^{(k,l)}$ 和 $delay_r^{(k,l)}$。变量 $delay_r^{(k,l)}$ 等于机器人 r 在执行完协同任务 ct_l^k 后，耗费的所有协同等待时间之和（包含之前执行协同任务时所耗费的等待时间）。在初始化时，这些变量的取值为 0。我们有以下约束：

$$t_r^{(k,l)} = \sum_{i \in Pre(j)} y_r^{(i,j)} \cdot (t_r^{(k',l')} + \omega_{\overline{P}}(q_i, q_j)), \tag{4-5}$$

其中，$ct_{l'}^{k'}$ 是集合 $\widetilde{\mathbb{T}}_r$ 中协同任务 ct_l^k 的前一项任务；$q_i \in \mathcal{C}(ct_{l'}^{k'})$ 且 $q_j \in \mathcal{C}(ct_l^k)$，根据定

中国知网 https://www.cnki.net

义 4.3 可知，存在边 $(q_i, q_j) \in \rightarrow_{\overline{P}}$，且 $\omega_{\overline{P}}(q_i, q_j)$ 表示机器人在这两个状态之间的转移时间。变量 $t_r^{(k,l)}$ 实际上表示了机器人 r 在不考虑协同等待时间的情况下，到达协同任务 ct_l^k 对应区域的理想时间。需要注意的是，由于 $\widetilde{\mathbb{T}}_r$ 中的第一项协同任务没有前继的元素，此时只考虑第二项 $\omega_{\overline{P}}(q_i, q_j)$ 的取值。

此外，我们定义变量集合 $\mathcal{Z} = \{z_l^k | \exists\, ct_l^k \in \widetilde{\mathbb{T}}, z_l^k = \max_{r\in\mathcal{N}}(t_r^{(k,l)} + delay_r^{(k',l')})\}$ 来表示对每一项协同任务，所有到达该任务区域的机器人中的最晚到达时间。如前所述，此处 $delay_r^{(k',l')}$ 表示机器人 r 在执行完协同任务 $ct_{l'}^{k'}$ 后的所有等待时间之和。进而有以下约束：

$$delay_r^{(k,l)} = z_l^k - t_r^{(k,l)}, \forall\, ct_l^k \in \mathbb{T}_r. \tag{4-6}$$

最终，MILP 策略优化模型的目标函数定义为：

$$J = \sum_{r\in\mathcal{N}} t_r^{(k_e,l_e)} + delay_r^{(k_e,l_e)}, \tag{4-7}$$

其中，k_e, l_e 对应于 $\widetilde{\mathbb{T}}_r$ 中最后一项协同任务 $ct_{l_e}^{k_e}$ 的右上标和右下标。

如前文所述，在求解 MILP 模型之后，我们即可根据集合 $\mathcal{Y}_r$ 中变量的取值获得每个机器人的最优执行策略。

时间复杂度：MILP 策略优化模型中的变量个数为 $O(N \cdot \Delta^2 \cdot |\widetilde{\mathbb{T}}_r| + |\widetilde{\mathbb{T}}|)$。对于每个机器人 $r \in \mathcal{N}$，进行状态缩减之后的个体乘积自动机 $\overline{\mathcal{P}}_r$ 具有分层结构，其层数为 $O(|\widetilde{\mathbb{T}}_r|)$。每一层包含了协同任务 $ct \in \widetilde{\mathbb{T}}_r$ 在 $\mathcal{P}_r$ 中对应的协同状态，其数量规模表示为 Δ。$\overline{\mathcal{P}}_r$ 中的边数为 $O(\Delta^2|\widetilde{\mathbb{T}}_r|)$，对应的集合 $\mathcal{Y}_r$ 中的变量个数也为 $O(\Delta^2|\widetilde{\mathbb{T}}_r|)$。另外，变量 $t_r^{(k,l)}$ 和 $delay_r^{(k,l)}$ 的个数等于 $|\widetilde{\mathbb{T}}_r|$，集合 $\mathcal{Z}$ 中的变量个数等于 $|\widetilde{\mathbb{T}}|$。

4.3 基于分布式迭代调整的执行策略优化

在上节中，我们基于状态缩减后的机器人个体乘积自动机建立了 MILP 执行策略优化模型，并得到了每个机器人 $r \in \mathcal{N}$ 的最优执行策略。然而，在仿真实验中，上述 MILP 模型随着机器人数量的增长，其求解效率迅速下降，无法满足多机器人的快速执行策略优化要求。这是由于 MILP 模型需要同时考虑求解并优化所有机器人的执行策略，随着机器人数量增加其复杂度迅速增长。

中国知网 https://www.cnki.net

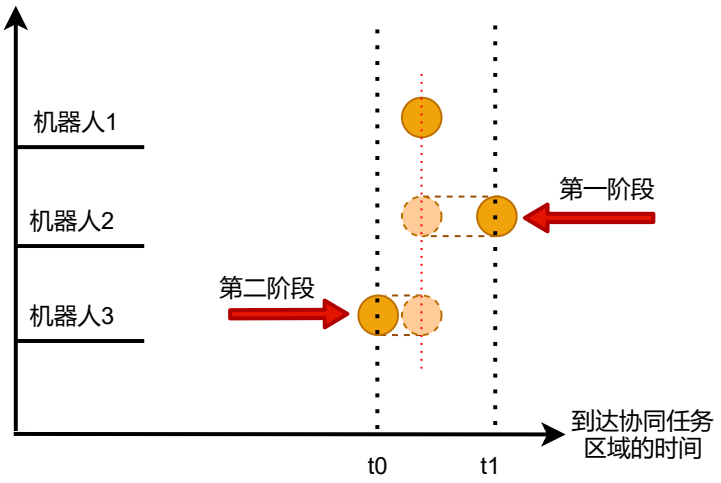


图 4.4: 机器人执行策略迭代调整机制示意图

在本小节，我们考虑分离执行策略求解与优化两个计算步骤，即对已有的机器人执行策略进行迭代地优化，来提升多机器人策略优化的效率。针对由于机器人到达协同任务区域时间不同而导致的协同等待时间，一种直观的调整算法如图 4.4 所示。通过提前最晚到达协同任务区域的机器人的到达时间，可以降低其它机器人在该协同任务中的协同等待时间；而通过推迟最早到达协同任务区域的机器人的到达时间，则可以降低该机器人自身的协同等待时间。在本节中，我们基于上述思想，提出一种分布式的执行策略迭代调整机制，来迭代地优化机器人的总时间损耗。该迭代调整机制考察能否通过调整机器人到达协同任务区域的时间，来减少多机器人总的任务执行时间。这一调整过程受启发于如何去减少机器人在执行协同任务时的低效等待时间。

图 4.5 展示在一次包括 5 个机器人的仿真实验中，执行策略调整前（透明圆盘）和调整后（实心圆盘），机器人到达被分配的协同任务区域的时间。另外，红色五角星表示了多机器人在执行策略调整前（透明）和调整后（实心）的完成任务耗时，这里考虑了它们在协同任务处的协同等待时间。全局协同任务 LTL_f 表达式为 $\phi = (\Diamond(\pi^{ct_1} \wedge (\Diamond\pi^{ct_2}))) \wedge (\Diamond\pi^{ct_3}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_1}\ \mathrm{U}\ (\Diamond\pi^{ct_6})) \wedge (\Diamond\pi^{ct_7}) \wedge (\Diamond\pi^{ct_8}) \wedge (\neg\pi^{ct_5}\ \mathrm{U}\ \pi^{ct_8})$。图中机器人到达每一项协同任务区域的时间的变化验证了本文所提出的执行策略迭代调整机制在降低多机器人执行任务总耗时方面的效果。上述执行策略迭代调整机制通过机器人之间的通信，分布式调整自身的执行策略，具体如算法 1 所示。需要说明的是，我们假设机器人之间的通信网络是连通的，因此信息可以传达到每一个机器人。

算法 1 主要分为三个阶段，包括初始化，通信与调整，终止。下面将详细阐述每一个阶段的算法内容。

中国知网 https://www.cnki.net

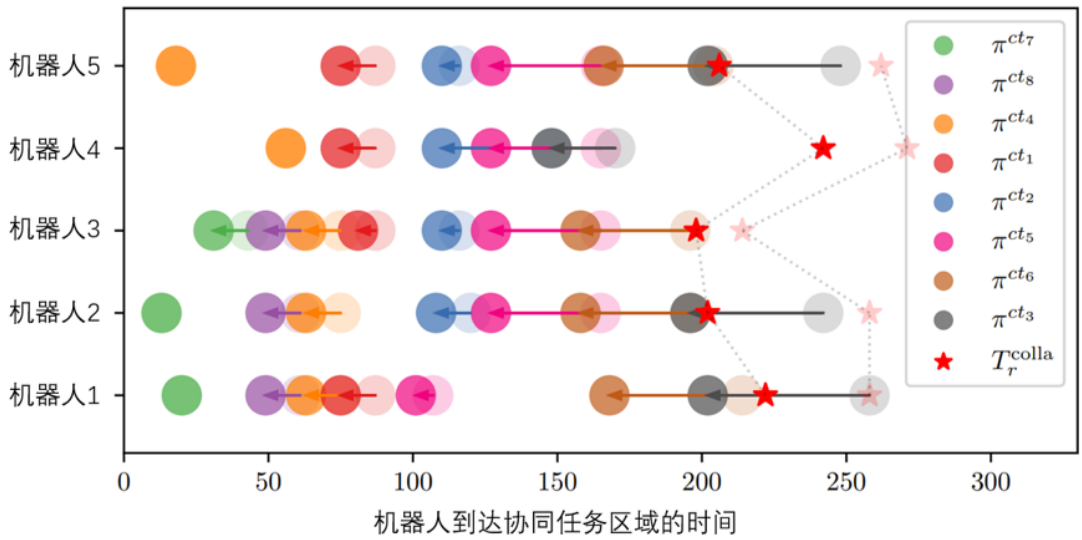


图 4.5: 机器人执行策略迭代调整机制的优化效果

算法 1 StrategyAdjustingMechanism

procedure 初始化:
  $count \leftarrow 0$, $ct \leftarrow \widetilde{\mathbb{T}}^{sort}$ 中第一项协同任务。
  $tl_r \leftarrow$ ComputeTimeline(r)，$\forall r \in \mathcal{N}$。
  每个机器人 $r \in \mathcal{N}$ 将 tl_r 发送给其它机器人。
  $msg \leftarrow \langle False, _, _, _, ct, count \rangle$。
  将 msg 发送给其它机器人。
end procedure

procedure 当机器人 r 收到 msg:
  $\langle success, r', received, tl_{r'}, ct, count \rangle \leftarrow msg$。
  if $success$ **then**
    将机器人 r 的 $tl_{r'}$ 更新为收到的 $tl_{r'}$。
  end if
  if $r \notin received \wedge r \in \mathcal{R}(ct) \wedge r = \text{FindLatest}(ct)$ **then**
    将机器人 r 添加到 $received$ 中。
    $can_opt \leftarrow$ AdjustStrategy(ct, $isLatest$ = true)。
    $ct^{next} \leftarrow$ct 在 $\widetilde{\mathbb{T}}^{sort}$ 中的下一项协同任务。
    if can_opt **then**
      $tl_r \leftarrow$ ComputeTimeline(r)。
      $msg \leftarrow \langle canOpt, r, \{\}, tl_r, ct^{next}, count + 1 \rangle$。
      将 msg 广播到其它的机器人。
    else
      $r' \leftarrow$ FindEarliest(ct)。
      $token \leftarrow \langle r, r', msg \rangle$。
      将 $token$ 传递到机器人 r'，即最早到达 ct 任务区域的机器人。
    end if
  else
    将 msg 发送给不在集合 $received$ 中的机器人。

中国知网 https://www.cnki.net

```
    end if
end procedure

procedure 当机器人 r 收到 token:
    ⟨r, r′, msg⟩ ← token。
    if r = r′ then
        can_opt← AdjustStrategy(ct, isLatest=false)。
        ct^next ← 集合 T̃^sort 中 ct 的下一项协同任务。
        if can_opt then
            count ← count + 1。
            tl_r ← ComputeTimeline(r)。
        else
            if ct 是最后一项协同任务 then
                if count = 0 then
                    向其它机器人广播算法终止信号。
                    Terminate。
                else
                    count ← 0, ct^next ← 集合 T̃^sort 中的第一项协同任务。
                end if
            end if
        end if
        msg ← ⟨canOpt, r, {}, tl_r, ct^next, count⟩。
        将 msg 广播到其它机器人。
    else
        received.add(r)。
        将 token 传递给机器人 r′。
    end if
end procedure
```

算法 2 FindLatest(ct)

$ct^{pre} \leftarrow ct$ 在集合 $\widetilde{\mathbb{T}}_r$ 中的前一项协同任务。
$r \leftarrow \arg\max_{r \in \mathcal{R}(ct)} \{t(ct^{pre}) - t_r(ct^{pre}) + t_r(ct)\}$。
return r

算法 3 FindEarliest(ct)

$ct^{pre} \leftarrow ct$ 在集合 $\widetilde{\mathbb{T}}_r$ 中的前一项协同任务。
$r \leftarrow \arg\min_{r \in \mathcal{R}(ct)} \{t(ct^{pre}) - t_r(ct^{pre}) + t_r(ct)\}$。
return r。

中国知网 https://www.cnki.net

算法 4 ComputeTimeline(r)

1: $tl_r = [\,]$。
2: **for all** $ct \in \widetilde{\mathbb{T}}_r$ **do**
3: $\quad t_r(ct) = \sum_{j=1}^{j^{ct}-1} \omega_r(\rho_r(j), \rho_r(j+1))$。
4: $\quad tl_r.\text{append}(t_r(ct))$。
5: **end for**
6: $T_r = \sum_{j=1}^{|\rho_r|-1} \omega_r(\rho_r(j), \rho_r(j+1))$。
7: $tl_r.\text{append}(T_r)$。
8: **return** tl_r

算法 5 ComputeTimeCost

1: $delay_r \leftarrow 0$, for $r \in \mathcal{N}$。
2: **for all** $ct \in \widetilde{\mathbb{T}}^{sort}$ **do**
3: $\quad \hat{t}_r(ct) = t_r(ct) + delay_r$, for $r \in \mathcal{R}(ct)$。
4: $\quad t(ct) = \max_{r \in \mathcal{R}(ct)} \hat{t}_r(ct)$。
5: $\quad delay_r = t(ct) - t_r(ct)$, for $r \in \mathcal{R}(ct)$。
6: **end for**
7: $T_r^{\text{colla}} \leftarrow T_r + delay_r$, for $r \in \mathcal{N}$。
8: $T^{\text{colla}} = \sum_{r \in \mathcal{N}} T_r^{\text{colla}}$。
9: **return** T^{colla}

4.3.1 初始化

在初始化阶段，每个机器人首先计算自身参与到每一项被分配的协同任务的时间序列，称为“时间线”（timeline），并将其传播给其它机器人。机器人 r 的时间线表示为 tl_r，其构造方法见算法 4。时间线 tl_r 记录了机器人 r 根据当前执行策略 ρ_r，到达每一项被分配的协同任务区域的理想时间（第 $2-4$ 行，算法 4），即不考虑在这些任务区域的协同等待时间。另外，tl_r 还包括了机器人 r 执行完所有任务耗费的总时间，同样也不包括协同等待时间（第 $6-7$ 行，算法 4）。

当机器人 r 接收到其它机器人 $r' \in \mathcal{N} \backslash \{r\}$ 发出的 $tl_{r'}$ 后，该机器人 r 即可在本地计算出在考虑协同等待时间的情况下，自己到达每个协同任务区域的时间，以及完成任务的总耗时，见算法 5。在算法 5 中，机器人在本地可以算出所有机器人参与到集合 $\widetilde{\mathbb{T}}^{sort}$ 中每个协同任务的实际到达时间，即 $\hat{t}_r(ct)$ 表示了机器人 r 实际到达 ct 对应的任务区域的时间，包括该机器人在之前的任务协同中由于同步约束而产生的协同等待时间 $delay_r$。其中，$\widetilde{\mathbb{T}}^{sort}$ 表示对第 3.1.3 节中选定的协同任务命题序列 σ 中的协同任务按照顺序进行排列所组成的集合。需要注意的是，$delay_r$ 是随着协同任务的执行而迭代更新的，即机器人 r 的总协同等待时间 $delay_r$ 随着参与的协同任务的数量的增加而逐渐累

中国知网 https://www.cnki.net

积。另外，算法 5 还可以计算所有机器人的总时间消耗，记为 T^{colla}（第 $7-8$ 行），这将用来在执行策略调整阶段验证机器人执行策略的调整效果。

此外，中心服务器将构造一条消息 msg 并将其广播到所有的机器人，该消息将触发机器人开始调整自身的执行策略。具体而言，这条 msg 消息将使最后一个到达 $\widetilde{\mathbb{T}}^{sort}$ 中的第一项协同任务的机器人自动获得令牌（$token$），获得该令牌的机器人将有权限调整自己的执行策略。关于消息 msg 的定义将在下一小节中给出。

4.3.2 通信与迭代调整

在本阶段，机器人在迭代调整机制作用下将依次考察 $\widetilde{\mathbb{T}}^{sort}$ 中所有的协同任务，通过调整机器人到达当前协同任务区域的时刻，检查能否降低执行协同任务时的等待时间，并优化及所有机器人的总时间损耗。这一过程依赖于两种信息的传递，其数据结构定义如下：

- msg，广播信息。一个消息 msg 是一个元组 $\langle canOpt, r, received, tl_r, ct, count\rangle$，其中 $canOpt$ 是一个布尔变量，用来指示上一个机器人 r 是否优化了总时间损耗；r 是产生当前 msg 消息的机器人编号；tl_r 是机器人 r 更新之后的时间线；ct 是当前即将考察的协同任务；$count$ 统计了在对 $\widetilde{\mathbb{T}}^{sort}$ 的一次遍历中对总时间损耗的成功优化次数。
- $token$，定义传输信息。一个消息 $token$ 是一个元组 $\langle r, r', msg\rangle$，其中 r 和 r' 分别是发出和即将接收 $token$ 的机器人编号。

当机器人 r 收到从机器人 r' 发出的 msg 消息后，该机器人 r 检查自身是否是最晚到达协同任务 ct 的任务区域的机器人，这可以通过运行函数 `FindLatest()` (第 13 行，算法 1) 来进行。若机器人 r 不是最晚到达的，则机器人 r 将自己的编号加入到 msg 消息的 $received$ 集合中，并通过多机器人之间的通信网络将该更新后的 msg 消息发送给所有不在 $received$ 集合中的机器人。反之，若机器人 r 就是最晚到达 ct 的协同任务区域的机器人，则机器人 r 开始调整自身的执行策略，具体见算法 6（在后文将进行详细介绍）。机器人 r 运行完算法 6 之后，将获得一个布尔变量 $canOpt$，该变量指示了机器人 r 是否找到了一个备选执行策略，使得多机器人总时间损耗可以降低。若 $canOpt$ 为真，则机器人 r 将该备选执行策略作为自己的新的执行策略，并根据新的执行策略重新

中国知网 https://www.cnki.net

计算 tl_r。随后构造一个新的 msg 消息，其中包括了新计算的 tl_r，并将其广播出去（第 19 – 21 行，算法 1）。

否则，若 $canOpt$ 为假，则机器人 r 通过运行函数 `FindEarliest()` 找到最早到达当前协同任务 ct 的任务区域的机器人 r'。随后机器人 r 构造消息 $token = \langle r, r', msg \rangle$ 并通过通信网络将该消息定向发送到机器人 r'。注意，若出现 $r = r'$，则机器人 r 不再进行这一步骤，而直接构造一个新的 msg 信号，并将其广播给其它机器人。需要指出的是，尽管当前机器人 r 可以根据本地保存的其它机器人的时间线 $\{tl_{r'}\}_{r' \in \mathcal{N} \setminus \{r\}}$ 获知哪一个机器人应当取得针对任务 ct^{next} 的令牌，msg 信息依然需要被广播给所有的机器人，以用于更新这些机器人本地保存的 tl_r，这里 ct^{next} 表示在 $\widetilde{\mathbb{T}}^{sort}$ 中协同任务 ct 的下一项任务。

当某个机器人 r 收到 $token$ 消息后（意味着机器人 r 是最早到达当前协同任务 ct 区域的机器人），机器人 r 即拥有了调整自身执行策略的令牌。机器人 r 可以通过调整自身的执行策略，即在满足时序约束的条件下调整访问不同任务区域的顺序，验证能否降低自身在当前协同任务中的等待时间，以及多机器人的总时间损耗（第 32 – 39 行，算法 1）。这一调整过程与收到 $token$ 消息的最晚到达协同任务区域的机器人类似，详见算法 6。不同之处在于，若此时运行算法 6 返回的布尔变量 $canOpt$ 为假，则机器人 r 直接构造用于触发对 $\widetilde{\mathbb{T}}^{sort}$ 中的下一项协同任务进行优化的 msg 消息，并广播给其它机器人。上述分布式通信机制如图 4.6 所示。

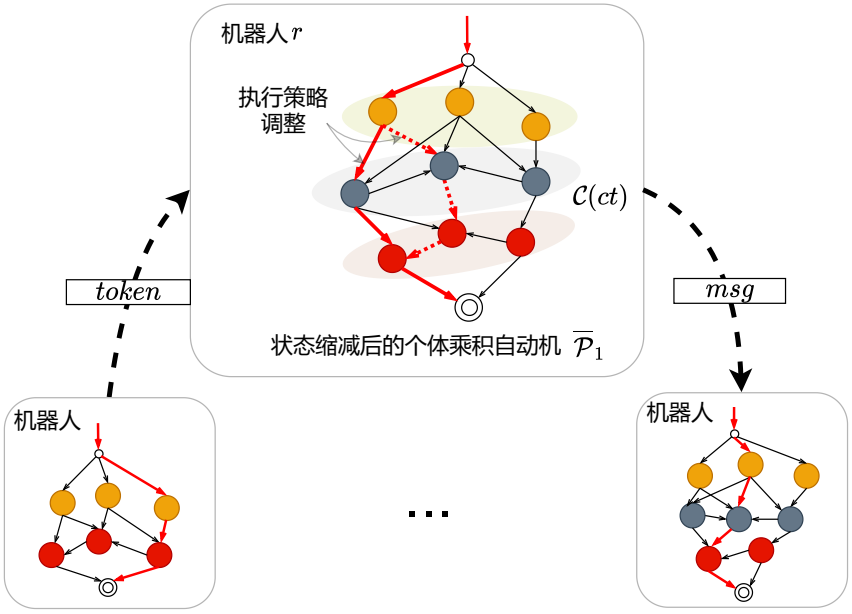


图 4.6: 执行策略迭代调整机制的分布式通信机制

中国知网 https://www.cnki.net

算法 6 AdjustStrategy($ct, isLatest$)

1: $ct^{pre} \leftarrow ct$ 在集合 $\widetilde{\mathbb{T}}_r$ 中的前一项协同任务。
2: **for all** $q \in \mathcal{C}(ct) \backslash \{\rho_r(j^{ct})\}$ **do**
3: $\quad \rho'_r \leftarrow \rho_r[..j^{pre}-1] + \text{Dijkstra}\big(\overline{\mathcal{P}}_r, \rho_r(j^{pre}), q\big) + \text{Dijkstra}(\overline{\mathcal{P}}_r, q, \mathcal{Q}_P^E)$
4: $\quad$ **if** $(isLatest \bigwedge t(ct^{pre}) - t_r(ct^{pre}) + t'_r(ct) < t_r(ct))$ or
5: $\quad\quad ((\neg\, isLatest) \bigwedge\ t_r(ct) < t(ct^{pre}) - t_r(ct^{pre}) + t'_r(ct) \leq t(ct))$ **then**
6: $\quad\quad\quad T_{\text{cand}}^{\text{colla}} \leftarrow \text{ComputeTimeCost}(\rho'_r)$。
7: $\quad\quad\quad$ **if** $T_{\text{cand}}^{\text{colla}} < T^{\text{colla}}$ **then**
8: $\quad\quad\quad\quad \rho_r \leftarrow \rho'_r$。
9: $\quad\quad\quad\quad$ **return** true
10: $\quad\quad\quad$ **end if**
11: $\quad$ **end if**
12: **end for**
13: **return** false

进一步的，我们对机器人调整自身执行策略的过程进行具体描述，见算法 6。假设机器人 r 获得了针对某个协同任务 $ct \in \widetilde{\mathbb{T}}_r$ 的令牌。$\overline{\mathcal{P}}_r$ 中的协同状态集合 $\mathcal{C}(ct)$ 实际上对应了机器人 r 执行任务 ct 的所有可选的状态，即对应着所有可能的到达 ct 任务区域的时间。机器人 r 随机遍历所有状态 $q \in \mathcal{C}(ct) \backslash \{\rho_r(j^{ct})\}$ 直到找到某个协同状态 q 可以优化 T^{colla}。对于每一个可选的协同状态 q，机器人 r 在 $\overline{\mathcal{P}}_r$ 上搜索一条待选路径 ρ'_r：首先保留机器人 r 在 $\overline{\mathcal{P}}_r$ 上的原始接收路径的前一部分不变，直到 $\rho_r(j^{pre}-1)$，即 $\rho_r(1)...\rho_r(j^{pre}-1)$；随后从状态 $\rho_r(j^{pre}-1)$ 开始搜索经过状态 q 到 $\overline{\mathcal{P}}_r$ 的接收状态的一条最短路径（第 $2-3$ 行，算法 6）。此处状态 $\rho_r(j^{pre})$ 是协同任务 ct^{pre} 在 $\overline{\mathcal{P}}_r$ 中的协同状态，ct^{pre} 是 ct 在 $\widetilde{\mathbb{T}}_r$ 中的前一项任务（第 1 行）。若以下两个条件成立，则上述待选路径 ρ'_r 将被作为机器人 r 新的接收路径 ρ_r（第 $4-9$ 行，算法 6）：

(a) 若机器人 r 是最晚到达当前协同任务区域的机器人，则有 $t(ct^{pre}) - t_r(ct^{pre}) + t'_r(ct) < t_r(ct)$; 若机器人 r 是最早到达当前协同任务区域的机器人，则有 $t_r(ct) < t(ct^{pre}) - t_r(ct^{pre}) + t'_r(ct) \leq t(ct)$。

(b) $T_{\text{cand}}^{\text{colla}} < T^{\text{colla}}$，其中 $T_{\text{cand}}^{\text{colla}}$ 表示当把 ρ_r 替换为 ρ'_r 后的多机器人总的时间损耗。

条件 (a) 保证了在待选接收路径 ρ'_r 中，最晚的机器人 r 到达协同任务 ct 的时间被提前了（或对于最早的机器人 r 而言被推迟）, 因此其它参与到 ct 的机器人等待时间将降低（或机器人 r 自己的等待时间被降低）。算法 6 中第 $4-5$ 行中的 $t'_r(ct)$ 是根据待选接收路径 ρ'_r 而计算的。条件 (b) 进一步保证了将待选路径 ρ'_r 作为机器人 r 新的接收路径也会使多机器人总时间损耗 T^{colla} 下降。这一限制使得机器人 r 在本地的贪心式（仅

中国知网 https://www.cnki.net

寻求在当前协同任务处的协同等待时间的减少）的策略调整同样也会对总体指标有所优化。若在遍历完所有的待选协同状态后，机器人 r 依然无法找到一个满足上述两个条件的可行的接收路径 ρ'_r，则算法返回 false。

4.3.3 终止条件

如果在对 $\widetilde{\mathbb{T}}^{sort}$ 中的所有协同任务的一次遍历中，未能成功对多机器人总时间损耗进行优化，即 $count = 0$，则上述迭代调整过程将结束。否则，$count > 0$ 即意味着机器人调整后的执行策略会使多机器人以更小的总时间损耗 T^{colla} 完成任务，随后迭代调整过程将会继续进行，重新对 $\widetilde{\mathbb{T}}^{sort}$ 中的所有协同任务依次进行同样的遍历考察，从而进行对机器人执行策略的迭代调整。

如果最早到达 $\widetilde{\mathbb{T}}^{sort}$ 中最后一项协同任务的机器人 r 未能成功通过调整自身执行策略优化多机器人执行任务的总耗时，则算法会检查上述终止条件是否成立。若上述终止条件成立，则该机器人 r 将会终止运行算法 6，并将终止信号广播给其它机器人（第 $41-44$ 行）。

注 4.2. 算法 6 得到的待选接收路径中的状态转移都是必要的。这里不必要的状态转移意味着机器人可能为了拖延到达任务区域的时间，故意经过一些本来无需经过的区域。算法 6 中不存在这种情况，因为算法搜索的是从上一项协同任务对应的协同状态出发经过当前协同任务的协同状态的最短接收路径，不存在可以避免的区域转移。

4.4 本章小结

本章基于全局协同任务的分解与分配结果，以及机器人的个体任务时序逻辑表达式，提出了基于个体乘积自动机的机器人执行策略求解方法。该方法首先将机器人被分配执行的协同任务及其内部存在的时序约束，与机器人的个体时序任务表达式相融合，获得更新后的个体任务 LTL_f 表达式。随后基于该表达式对应的 NFA 与机器人个体的加权切换系统，构建个体乘积自动机，并通过在自动机上搜索最短接收路径获得满足所有任务要求的可行的机器人执行策略。

进一步，针对机器人参与协同任务时由于同步要求而产生的协同等待时间，本文提出一种基于分布式通信的机器人执行策略迭代调整机制，以优化机器人执行协同任务的

中国知网 https://www.cnki.net

时间效率，从而降低多机器人系统的总时间损耗。与基于混合整数规划模型的执行策略优化方法对比，该方法可以将计算量均摊到个体机器人本地，从而提高了执行策略优化的效率。

中国知网 https://www.cnki.net

5 多机器人时序任务规划算法的理论分析与实验验证

本章将对本文提出的多机器人时序任务规划算法框架进行理论分析以及实验验证，通过与常见的基于构建全局乘积自动机的经典算法进行对比，验证本文算法框架的求解效率与性能表现。另外，本章还通过对比基于混合整数规划模型的机器人执行策略求解算法，来展示执行策略迭代调整机制在求解时间以及优化效果方面的表现。

5.1 多机器人时序任务规划算法框架

结合第 3 章中的协同任务 LTL_f 表达式的分解与协同任务的分配，以及第 4 章中多机器人执行策略的求解与优化，本文提出的多机器人时序任务规划框架总述为算法 7。

算法 7 多机器人时序任务规划框架

输入: $\{\varphi_r\}_{r\in\mathcal{N}}, \phi, \mathcal{W}, \widetilde{\mathbb{T}}, \{\overline{\mathbb{T}}_r\}_{r\in\mathcal{N}}$
输出: $\{\tau_r\}_{r\in\mathcal{N}}$

```
对每个机器人 r ∈ N，构建加权切换系统 wTS_r。
将表达式 φ 转化为对应的 NFA 模型 F。
筛除 F 中不可行的状态转移条件，并识别 F 中的分解状态。
从 F 中选择最短路径 ρ_F 并提取对应的关键序列 σ，利用分解状态将其分解为相互独立的协同任务的子命题序列 σ = σ^1; σ^2; ...; σ^S。
构建基于 SMT 的协同任务分配模型 f。
while SAT(f) do
    X ← SMT-solver(f)。
    将 X 发送给机器人。
    for r ∈ N do
        构建 LTL_f 表达式 φ_r，描述分配给机器人 r 的协同任务 ct ∈ T̃_r 的时序约束。
        φ̃_r ← φ_r ⋀ φ_r。
        将 φ̃_r 转化为对应的 NFA 模型 F_r。
        P_r ← wTS_r ⨂ F_r。
        ρ_r ← P_r 的最短接收路径。
    end for
    构造状态缩减后的个体乘积自动机 P̄_r, ∀r ∈ N。
    StrategyAdjustingMechanism ({P̄_r}_{r∈N}, {ρ_r}_{r∈N})。
    f ← f ⋀ ¬X。
end while
return 对每个机器人 r ∈ N，返回 τ_r ← Π_r ρ_r。
```

该算法框架的流程如图 5.1 所示，包括协同任务独立性分解，协同任务分配，个体

中国知网 https://www.cnki.net

机器人执行策略求解与分布式执行策略迭代调整等四个阶段:

- 在协同任务独立性分解阶段，中心服务器首先将机器人的协同任务 LTL_f 表达式 ϕ 转化为对应的 NFA 模型 $\mathcal{F}$，然后根据第 3.1 章的内容，在 NFA 模型上搜索一个满足表达式 ϕ 的协同任务序列 σ，并根据分解状态将其分解为可以独立执行的子序列（算法 7，第 2 – 4 行）。
- 在协同任务分配阶段，通过建立 SMT 约束模型，将协同任务序列 σ 中的每一项协同任务分配给机器人，以满足协同任务要求以及任务区域之间的通信约束等，并利用快速筛除机制过滤非最优的协同任务分配方案（第 5 – 7 行）。
- 在个体机器人执行策略求解阶段，每个机器人获得当前协同任务分配方案 X 后，在本地将分配到的协同任务与个体任务 LTL_f 表达式相结合，并与自身加权切换系统结合来求解自身的执行策略（第 9 – 15 行）。
- 在分布式执行策略迭代调整阶段，机器人可以通过调整自身执行策略来进一步降低机器人在参与协同任务时的等待时间，从而提升完成任务的整体时间效率（第 16 – 17 行）。

另外，在求解时间允许的情况下，该算法框架将迭代遍历所有可行的协同任务分配方案，以持续优化多机器人执行策略的实际效率（第 6, 18 行）。

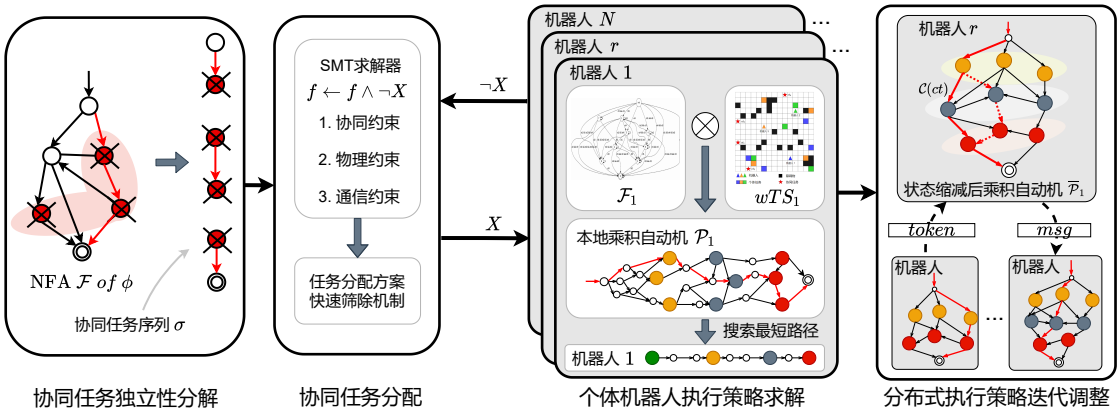


图 5.1: 多机器人时序任务规划框架流程图

5.2　完备性、复杂度与最优性分析

命题 5.1. 算法 7 是完备的。

中国知网　https://www.cnki.net

证明. 此证明基于命题 4.1。注意在执行策略迭代调整阶段，机器人只会在自身状态缩减后的个体乘积自动机上搜索另一条接收路径，这条路径必定对应于状态缩减前的个体乘积自动机中的一条接收路径。上述路径的改变并不会影响命题 4.1，因此利用与命题 4.1 同样的证明方法，我们可以说明基于与命题 4.1 同样的假设，本文所提出的多机器人时序任务规划框架在执行策略的迭代调整下依然是完备的。 □

命题 5.2. 在执行策略迭代调整机制中，多机器人完成任务的总时间损耗会单调降低，且该迭代调整机制将会在有限的迭代次数内终止。

证明. 在给定的协同任务分配方案下，根据算法 6 的第 $7-8$ 行，T^{colla} 将从 $T^{\text{colla}}_{\text{init}}$ 单调下降直到算法终止，其中 $T^{\text{colla}}_{\text{init}}$ 是在进行执行策略迭代调整前多机器人的总时间损耗。另外，T^{colla} 不会比 T^{indiv} 更小，T^{indiv} 代表了在不考虑协同等待时间的情况下多机器人总时间损耗。考虑到环境模型中的状态转移时间的最小单位是有限的，因此可以得出算法 6 将会在有限的迭代次数内终止。 □

命题 5.3. 算法 6 的最差时间复杂度是 $\mathcal{O}\left((T^{\text{colla}}_{\text{init}} - T^{\text{indiv}}) \cdot |\overline{\mathcal{P}}_r| \cdot (E \cdot \lg E + |\sigma| \cdot N)\right)$，其中 N 代表机器人的数量，$|\overline{\mathcal{P}}_r|$ 是个体乘积自动机的状态数的最大值，E 是所有机器人 $r \in \mathcal{N}$ 的 $\overline{\mathcal{P}}_r$ 中的边数的最大值。$|\sigma|$ 是在第 3.1.3 章中选择的接收路径 σ 的长度。

证明. 算法 6 中，拥有令牌的机器人在尝试调整自身的执行策略时，在最差的情况下将会遍历 $\overline{\mathcal{P}}_r$ 中所有的待选协同状态。如命题 5.2 所述，调整的次数为 $\mathcal{O}(T^{\text{colla}}_{\text{init}} - T^{\text{indiv}})$。对于每一个待选的协同状态，拥有令牌的机器人 r 利用 Dijkstra 算法在 $\overline{\mathcal{P}}_r$ 中搜索一条路径 ρ'_r，其复杂度为 $\mathcal{O}(E \cdot \lg E)$，并且调用算法 5 来验证 ρ'_r 是否更优，其复杂度为 $|\sigma| \cdot N$。总结来看，算法 6 的最差时间复杂度为 $\mathcal{O}\left((T^{\text{colla}}_{\text{init}} - T^{\text{indiv}}) \cdot |\overline{\mathcal{P}}_r| \cdot (E \cdot \lg E + |\sigma| \cdot N)\right)$。

□

非最优性分析: 本文所提出的多机器人时序任务规划框架不保证可以找到最优解。因为在执行策略迭代调整阶段，机器人贪心地调整自身的执行策略（只考虑优化当前协同任务处的协同等待时间，且一旦找到更优的执行策略就退出），而不是遍历所有可行的待选路径。在给定的协同任务分配方案下，MILP 模型可以获得最优的多机器人执行策略，但是该方法随着机器人数量的增长求解效率降低（见第 5.3 章实验结果）。另外，第 3.1.3 章中对协同任务序列 σ 的选择也会影响最终的执行策略的性能。

中国知网 https://www.cnki.net

5.3　算法可行性与求解效率验证

在本小节，我们通过仿真实验评估验证了所提方法的可拓展性，解质量以及求解效率。所有仿真实验均在一台 Ubuntu 16.04 服务器上运行，CPU 型号为 Intel Xeon E5-2660 (2.00GHz)，RAM 为 128 GB。我们使用 Z3 求解器[66]求解在协同任务分配阶段建立的 SMT 模型。基于 MILP 的策略优化模型利用 Gurobi V9[65]求解。仿真实验的代码利用 Python 3 编写，并利用软件包 igraph [1]来建立程序中的图模型并进行相应的操作。该软件包基于 C 语言编写，在进行图的构建与修改等操作时效率较高。

我们假设所有的个体任务和协同任务都随机分布在一个网格地图中。每个机器人 r 有一个个体任务 LTL_f 表达式 φ_r，例如，$\varphi_r = (\Diamond(\pi^{ts_1} \vee \pi^{ts_4})) \wedge (\Diamond\pi^{ts_2}) \wedge (\neg\pi^{ts_3}\ \mathrm{U}\ \pi^{ts_2}) \wedge (\Diamond(\pi^{ts_4} \rightarrow (\Diamond\pi^{ts_3})))$。另外，所有的机器人需要协同完成全局协同任务 LTL_f 表达式 ϕ，例如，$\phi = (\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\ \mathrm{U}\ \pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \rightarrow (\Diamond\pi^{ct_3})))$，该表达式对应有 $|\widetilde{\mathbb{T}}| = 4$.

在仿真中，我们假设协同任务 $ct \in \widetilde{\mathbb{T}}$ 的完成需要多个拥有集合 $\{c_1, c_2, c_3\}$ 中的技术功能的机器人的协同，并随机生成这些协同任务要求的各类机器人的数量。

5.3.1　多机器人时序任务规划框架的可行性

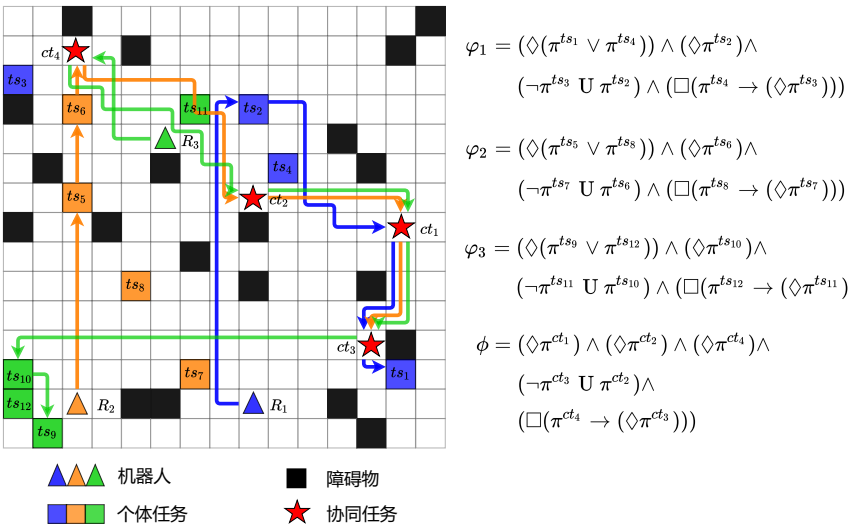


图 5.2: 图 2.2所示仿真案例中的机器人执行策略

本小节中，我们将通过一个包含 3 个机器人的仿真案例来说明多机器人时序任务

[1]https://igraph.org/python/

中国知网　https://www.cnki.net

规划框架的求解可行性。该仿真案例的问题设定与第 2 章中图 2.2 相同。利用本文提出的多机器人时序任务规划算法求得的机器人执行策略如图 5.2 所示。就机器人 1 而言，可以看到该机器人执行个体任务的顺序为 $ts_2\,ts_1$，这满足个体任务 LTL_f 表达式 $\varphi_1 = (\Diamond(\pi^{ts_1} \vee \pi^{ts_4})) \wedge (\Diamond\pi^{ts_2}) \wedge (\neg\pi^{ts_3}\,\mathrm{U}\,\pi^{ts_2}) \wedge (\Box(\pi^{ts_4} \rightarrow (\Diamond\pi^{ts_3})))$ 的要求，即：在任务 ts_1 和 ts_4 之间，机器人将执行 ts_1；由于不执行 ts_4，因此不要求执行 ts_3；ts_1 和 ts_2 的执行之间不存在时序约束。就协同任务要求的角度来看，协同任务被执行的顺序为 $ct_4\,ct_2\,ct_1\,ct_3$，该顺序满足协同任务 LTL_f 表达式 $\phi = (\Diamond\pi^{ct_1}) \wedge (\Diamond\pi^{ct_2}) \wedge (\Diamond\pi^{ct_4}) \wedge (\neg\pi^{ct_3}\,\mathrm{U}\,\pi^{ct_2}) \wedge (\Box(\pi^{ct_4} \rightarrow (\Diamond\pi^{ct_3})))$ 的要求，即：ct_1，ct_2，ct_4 都被执行；ct_3 在 ct_2 之后执行；ct_4 执行之后引发了 ct_3 的执行。另外，从图 5.2 中可以看出，机器人在任意两个任务区域之间的转移路径均为最短路径，这体现出了基于模型检查工具的时序逻辑规划方法可以同时满足任务之间的时序逻辑约束以及搜索最优路径的功能优势。

上述小规模仿真案例验证了本文所提出的机器人时序任务规划算法的可行性。

5.3.2 多机器人时序任务规划框架的可拓展性

我们将提出的算法与基于全局乘积自动机的经典算法（称为 baseline 方法）进行了比较，并利用文献 [58] 所提出的状态缩减方法来提升经典算法在求解该问题时的性能。具体而言，每个机器人构造一个本地乘积自动机 $\dot{\mathcal{P}}_r = wTS_r \otimes \dot{\mathcal{F}}_i$，其中 wTS_r 中只保留对应于任务区域的状态，而删除其它与任务执行的无关状态，因为无关状态所对应的区域没有需要完成的任务。$\dot{\mathcal{F}}_i$ 是 LTL_f 表达式 φ_r 对应的 NFA。随后，我们建立一个全局乘积自动机 $\ddot{\mathcal{P}} = \dot{\mathcal{P}} \otimes \mathcal{F}$，其中 $\dot{\mathcal{P}} = \dot{\mathcal{P}}_1 \otimes \cdots \otimes \dot{\mathcal{P}}_N$，且 $\mathcal{F}$ 是 LTL_f 表达式 ϕ 对应的 NFA。注意，此处的符号 $\dot{\mathcal{F}}$，$\dot{\mathcal{P}}$ 和 $\ddot{\mathcal{P}}$ 上方的圆点是用于与本文中我们提出的方法所用的符号加以区分。构建全局乘积自动机后，多机器人的执行策略可以通过在该自动机 $\ddot{\mathcal{P}}$ 中搜索一条最短接收路径来获得。需要指出的是，该最短路径不能保证多机器人的总时间损耗最小，因为在路径搜索过程中并未考虑多机器人的协同等待时间，而假设所有机器人都是同步进行状态转移的。关于如何在 $\ddot{\mathcal{P}}$ 中搜索总时间损耗最小的路径（考虑协同等待时间），依然是一个需要解决的问题。本文中直接取最短路径作为经典算法获得的最终结果。尽管存在上述问题，但这对我们评估对比本文所提出的算法与经典算法的可拓展性影响较小。我们评估了这两种方法在不同的任务设置、机器人数量的情况下的性能表现，同时固定环境的大小为 30×30 的网格地图，仿真结果如表 5.1 所示。

中国知网 https://www.cnki.net

表 5.1: 基于全局乘积自动机的方法（Baseline）对比多机器人时序任务规划算法（Proposed）

N	$\lvert\widetilde{\mathbb{T}}\rvert$	$\lvert\ddot{\mathcal{P}}\rvert$	Baseline (秒/秒)	Proposed (秒/秒)	
				第一个可行解	最优解
2	4	34848	266/12.4	252/11.5	252/47.5
	6	161376	338/97.0	352/28.2	352/57.6
3	4	3241792	333/41962.6	342/19.8	330/195.1
	6	1.7×10^7	-/-	390/66.2	326/1800
5	6	1.4×10^{11}	-/-	819/52.9	802/1800
50	6	5.2×10^{97}	-/-	13702/700.4	13702/1800
100	6	2.7×10^{195}	-/-	26514/3548.7	26514/3548.7

Baseline 和 Proposed 对应的列的数据结构为：$T^{\text{colla}}/t^{\text{cal}}$，其中 T^{colla} 是多机器人总时间损耗，t^{cal} 是求解时间。记号 “-” 表明相应实例求解过程中电脑内存溢出。

针对本文提出的方法，我们记录了在 30 分钟的求解时间内，该方法在不同的问题实例下获得第一个可行解的耗时，以及在 30 分钟内的最优解。从表 5.1 中可见，基于全局乘积自动机的 baseline 方法随着机器人数量 N 的增长和协同任务数量 $|\widetilde{\mathbb{T}}|$ 的增长而迅速变得难以求解。而本文所提出的方法，其求解时间大致与机器人数量呈线性增长。这是由于在统计求解时间时我们叠加了机器人构造个体乘积自动机的时间（而构造自动机占用了求解过程中的大部分时间）。可以看出，当把个体乘积自动机的构造过程分散到每个机器人执行时，本文提出的方法求解时间将会进一步下降，并在大规模机器人的应用中具有良好的拓展性。这一实验结果表明本文提出的方法可以快速获得可行的多机器人执行策略，且与 baseline 方法对比，有很高的可拓展性。产生这一现象的原因是，本文所提出的规划框架中，协同时序任务要求被分配到个体机器人身上，因此只需要通过个体机器人的本地计算即可求解出满足任务要求的执行策略，即计算复杂度不会因机器人数量增长而出现明显变化。在表 5.1 中本文提出的算法求解时间大致与机器人数量呈线性增长，这是由于我们累加了所有机器人的本地求解时间。而基于全局乘积自动机的方法的计算复杂度与机器人数量呈指数增长，因此在机器人规模增大时变得难以求解。

5.3.3 执行策略迭代调整机制的求解效率与优化性能

在本小节中，我们进行了多组仿真实验来考察本文所提出的策略迭代调整机制的策略优化效果，并与基于 MILP 策略优化模型的方法进行对比。我们设置了多种机器人数

中国知网 https://www.cnki.net

量，包括 $\{5, 10, 15, 20, 25, 30\}$，也考察了多机器人分别在 4, 6 和 8 项等不同数量的协同任务下的性能表现。另外，我们还考察了在三种不同的环境规模下的实验结果。在每一种仿真实例中，我们将本文中所提出的多机器人时序任务规划方法（包括策略迭代调整机制）运行 10 次，其中每一次机器人和任务都被随机分布在工作环境中。对于本文所提出的方法，当求解时间超过 30 分钟时，我们就停止算法运行，并选取到停止时得到的多机器人的最佳执行策略。在相同的仿真实例下，我们比较了本文提出的方法与 MILP 方法在求解时间、解的质量方面的性能表现。

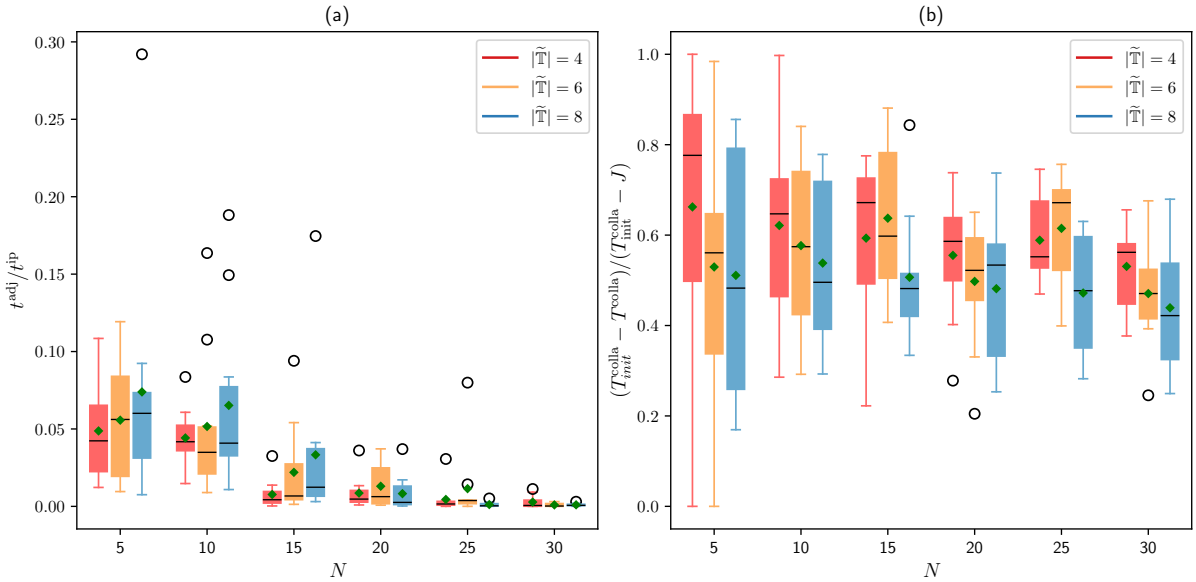


图 5.3: 不同数量的协同任务下 MILP 方法与迭代调整机制的性能比较

在图 5.3(a) 中，我们比较了策略迭代调整算法在不同的机器人数量以及不同数量的协同任务下，与 MILP 方法的求解时间的比值。图中展示了在每种仿真实例下，10 次实验中的平均值（绿色菱形），中位数（黑色横线），第 25 及第 74 个百分位点，以及异常值（圆圈）。明显可以看出，在所有的仿真实例中，本文所提出的策略迭代调整机制的求解时间均远小于 MILP 方法，仅需 MILP 方法约 10% 的即可完成求解。并且，随着机器人数量的增长，策略迭代调整机制与 MILP 方法的求解时间的比值变得更小，表明前者在求解效率方面的优势相较后者在逐渐扩大。关于本文所进行的实例的具体仿真数据统计见表 5.2。另外，在图 5.3(b) 中，我们评估了本文提出的执行策略迭代调整机制相对与 MILP 方法所求得的机器人执行策略在总时间损耗上的优化程度。优化程度的评估指标设定为 $(T_{\text{init}}^{\text{colla}} - T^{\text{colla}})/(T_{\text{init}}^{\text{colla}} - J)$，其中 $T_{\text{init}}^{\text{colla}}$ 表示在运行执行策略迭代调整机制

中国知网　https://www.cnki.net

之前，机器人的初始执行策略所对应的多机器人总时间损耗，T^{colla} 是在执行策略迭代调整机制之后的多机器人总时间损耗，J 表示 MILP 方法求得的最优执行策略的总时间损耗。从图 5.3(b) 中可以看出，当多机器人的规模较小时，本文所提出的策略迭代调整算法的优化效果差异较大，但随着机器人数量的增长，优化程度逐渐稳定在约 40% 到 60% 之间，这验证了该算法在降低多机器人总时间损耗方面的效果。

表 5.2: 状态缩减时间、策略迭代调整机制求解时间以及 MILP 方法求解时间对比

$\lvert\widetilde{\mathbb{T}}\rvert$	单位: 秒	N					
		5	10	15	20	25	30
4	$\bar{t}^{\text{prune}}$	8.14 ± 3.93	13.70 ± 3.12	12.87 ± 2.74	10.90 ± 2.99	8.32 ± 3.65	5.17 ± 2.05
	t^{adj}	0.07 ± 0.04	0.27 ± 0.08	0.35 ± 0.15	0.41 ± 0.21	0.33 ± 0.18	0.38 ± 0.21
	t^{ilp}	2.05 ± 1.53	7.21 ± 3.37	181.18 ± 229.33	146.72 ± 148.56	1741.57 ± 4160.64	4129.20 ± 6706.97
6	$\bar{t}^{\text{prune}}$	30.49 ± 5.47	22.16 ± 7.16	25.74 ± 5.74	25.00 ± 7.36	22.79 ± 9.21	17.31 ± 6.97
	t^{adj}	0.17 ± 0.12	0.26 ± 0.16	0.56 ± 0.28	0.66 ± 0.37	0.60 ± 0.31	0.60 ± 0.28
	t^{ilp}	3.78 ± 1.89	8.31 ± 6.87	91.24 ± 100.05	261.13 ± 358.53	808.92 ± 1454.72	4941.11 ± 5694.07
8	$\bar{t}^{\text{prune}}$	48.59 ± 14.38	38.53 ± 13.04	38.43 ± 13.08	29.25 ± 12.54	28.71 ± 15.82	30.05 ± 10.86
	t^{adj}	0.28 ± 0.16	0.36 ± 0.19	0.63 ± 0.26	0.45 ± 0.29	0.86 ± 0.51	0.87 ± 0.42
	t^{ilp}	6.43 ± 4.20	8.24 ± 5.33	77.86 ± 80.47	516.50 ± 1186.96	2816.32 ± 2164.77	4271.79 ± 8339.85

表 5.2 展示了在上述仿真实验中不同的机器人数量以及协同任务数量下的每一种实例对应的详细求解时间。表格中包含了：机器人按照定义 4.3 构建缩减状态后的个体乘积自动机的平均时间（$\bar{t}^{\text{prune}}$）；策略迭代调整机制所花费的时间（t^{adj}）；以及求解 MILP 模型所花费的时间（t^{ip}）。需要指出的是，机器人构建缩减状态后的个体乘积自动机是上述策略迭代调整机制和 MILP 方法两种方法的共同前置步骤，这使得我们可以直接对比 t^{adj} 和 t^{ip} 来对比这两种算法的时间效率。如表 5.2 中所示，$\bar{t}^{\text{prune}}$ 随着协同子任务数量的增多而增大，这与注 4.2 的分析是一致的。除了机器人较小时的一些实例，t^{adj} 和 t^{ip} 也具有同样的增大趋势。

由于状态缩减后的个体乘积自动机大幅降低了原始自动机的规模，减小了 MILP 优化模型的规模，因此 MILP 方法可以在机器人数量较小的时候在 10 秒钟之内求解出最优执行策略。但是，随着机器人数量的增长，t^{ip} 开始急剧增大。注意在 $N = 30$ 和 $\lvert\widetilde{\mathbb{T}}\rvert = 8$ 的实例中，MILP 方法耗费了超过 7 小时才得到最优解，而在同一实例中迭代调整机制仅耗时 1.93 秒就终止并得到了次优解。值得注意的是，在几乎所有的实例中，本文所提出的迭代调整机制都在 1 秒以内完成了求解（所有实例中的最大值为 2.24 秒）。

另外，我们还比较了两种策略优化方法在不同的环境规模下的上述性能表现，包括 20×20、30×30 和 40×40 网格地图等。实验结果如图 5.4 所示。两种算法的性能表现

中国知网 https://www.cnki.net

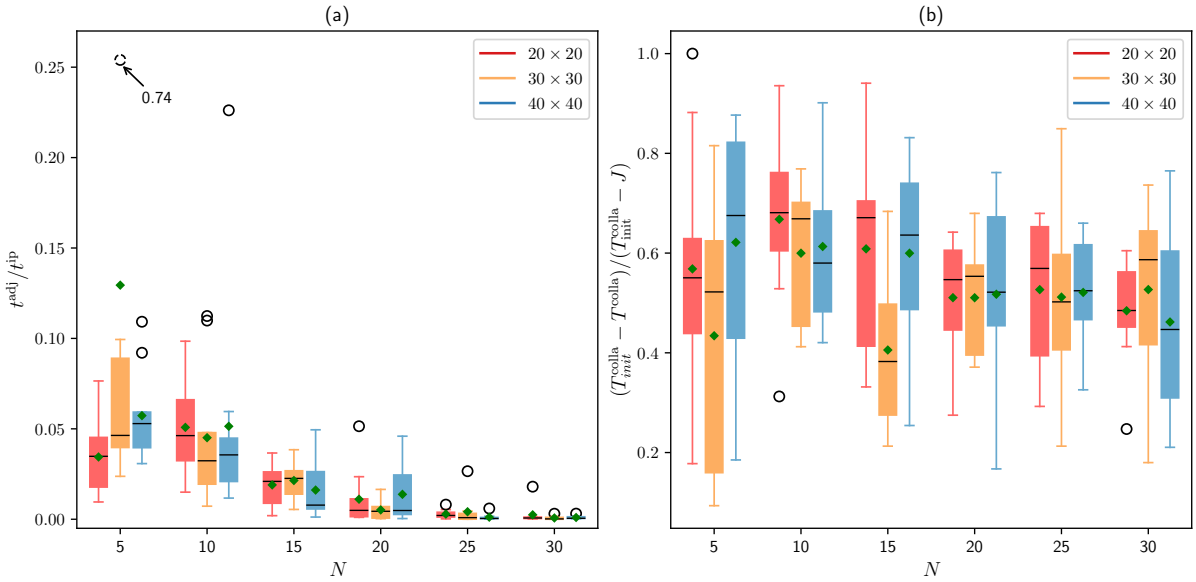


图 5.4: 不同环境规模下 MILP 方法与迭代调整机制的性能比较

基本与图 5.3 一致。可以看到，图 5.4 中迭代调整机制的性能表现并没有随着工作环境规模的变化而产生明显差异，这是因为工作环境的规模并不会影响状态缩减后的个体乘积自动机的规模，见注 4.2。工作环境的规模会影响机器人在不同任务区域之间的移动时间，而在图 5.4(*b*) 中我们可以看到，这一改变并未对迭代调整机制的优化效果产生明显影响。

5.3.4　任务分配方案的迭代评估与快速筛除效果

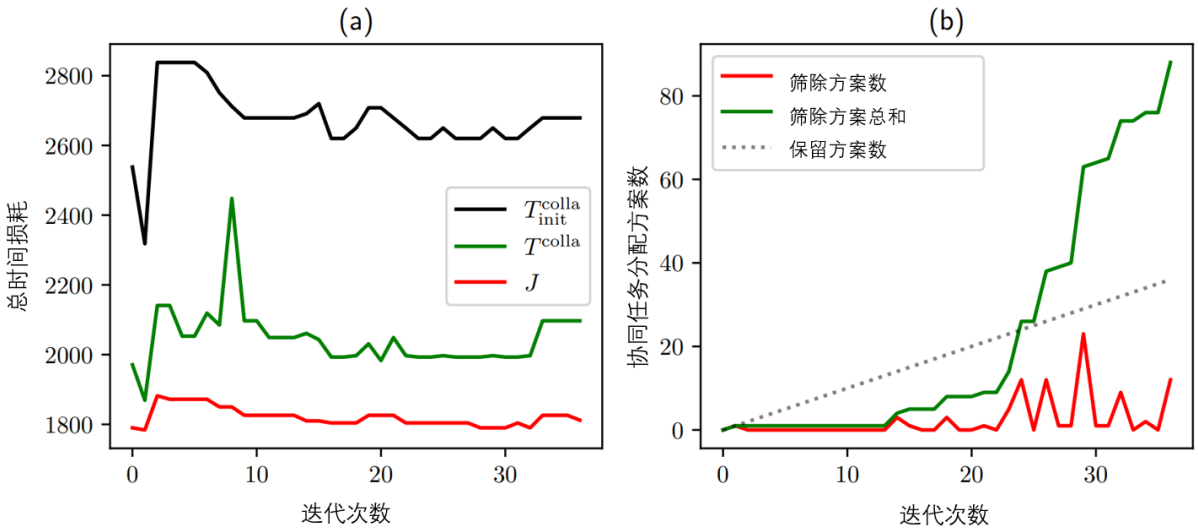


图 5.5: 不同协同任务分配方案下的执行策略优化过程以及快速筛除策略的效果

本文所提出的多机器人时序任务规划框架通过迭代的更新 SMT 模型约束并进行求

中国知网　https://www.cnki.net

解（见第 3.2 章），以迭代地评估每一个可行的协同任务分配方案。同时，我们提出了一种快速筛除机制，用于在进行后续任务规划之前预先筛除掉非最优的协同任务分配方案。图 5.5(a) 展示了在一次包括 15 个机器人，$|\widetilde{\mathbb{T}}| = 6$，工作环境规模为 30×30 的仿真实例中，每一种可能的协同任务分配方案下的求解结果。在每一次迭代中（每一种协同任务分配方案下），多机器人总时间损耗通过运行执行策略调整机制从 $T_{\text{init}}^{\text{colla}}$ 被优化到 T^{colla}。尽管在每一次迭代中 MILP 方法可以提供更优的执行策略，如图中红色折线所示，MILP 方法相较迭代调整机制却要耗费更多的时间来求解。另外，如图 5.5(b) 所示，在第 3.2 章中所提出的协同任务分配方案快速筛除机制可以有效筛除非最优的协同任务分配方案。该策略通过记录之前获得的可行的协同任务分配方案，来考察当前求得的任务分配方案是否最优，这避免了不必要的后续计算。这一筛除策略只需要维持一个与迭代次数成比例的集合，其中保存着之前考察过的未被筛除的任务分配方案。需要指出的是，筛除的效率取决于 SMT 求解器输出可行解的内部机制，以及协同任务要求的机器人类型和数量与多机器人的功能特点等。

5.4 本章小结

针对本文提出的多机器人时序任务规划框架，本章从理论分析和仿真实验两个方面分析并验证了该算法框架的求解效率优势以及实际执行效果。本章首先分析了该算法框架的完备性，以及执行策略迭代调整机制的收敛性，以及具体算法模块的计算复杂度。同时，本章客观分析了影响机器人执行策略最优性的因素，为后续算法的改进与提升提供了借鉴。

然后，本章进行了大量不同问题设置下的仿真实验，包括不同的机器人数量、环境规模以及任务要求，通过与基于全局乘积自动机的经典多机器人时序任务规划算法对比，验证了本文提出的算法框架在求解效率以及可拓展性方面的显著优势。本章还对比了基于混合整数规划模型的机器人执行策略求解方法，与本文提出的迭代调整机制在算法性能上的表现，仿真结果表明迭代调整机制可以快速实现对机器人执行策略的优化，并具有良好的时间效率提升效果。

中国知网 https://www.cnki.net

6 结论与展望

6.1 本文的主要研究成果

多机器人协同广泛存在于物流、军事、环境监测等多个社会领域。日益提高的社会智能化水平为多机器人系统提出了更为复杂的任务要求，如任务的完成需要多机器人的协同以及任务之间存在复杂的时序约束等。在本文中，我们聚焦于实际生产中的常见问题场景，即系统中的每个机器人具有自己的个体任务 LTL_f 表达式，同时多机器人还需要通过协作完成一个全局的协同任务 LTL_f 表达式，并研究如何高效求解机器人的任务执行策略并优化实际执行中的时间效率。

已有的研究工作通常基于于分布式的在线式规划或集中式的计算。其中，许多在线算法需要预先指定机器人之间的协同关系，且难以从全局的角度优化机器人的实际执行效率。而集中式方法在多机器人任务规划中存在计算复杂度高，可拓展性较差等缺陷。为了弥补当前研究工作中存在的问题，本文提出了一种多机器人时序任务规划框架，该框架无需提前指定机器人的协同关系，并在任务规划中将计算量分摊到个体机器人，有效提升了该规划框架的求解效率和可拓展性。另外，该规划框架允许机器人对自身执行策略进行迭代调整，来优化机器人执行协同任务时的时间效率。上述多机器人时序任务规划框架为多机器人系统应用于复杂任务场景中提供了有效的算法工具与一定的理论基础。

本文的主要研究成果如下：

（1）针对时序约束下的协同任务要求存在建模难，规划复杂度高等问题，本文利用线性时序逻辑语言来描述协同任务要求，基于自动机理论识别该协同任务要求中可以相互独立执行的部分并进行分解，从而降低实际执行中机器人之间的耦合程度；为进一步将任务规划过程分摊到个体机器人以降低整体规划问题的规模，本文建立了 SMT 模型来求解时序逻辑约束下的协同任务分配问题，并考虑通信受限的情况下如何满足协同任务之间的时序约束。此外，该方法利用 SMT 模型的迭代求解机制提出一种快速筛除策略，实现对协同任务分配方案的快速遍历与持续优化。

（2）针对机器人同时具有个体时序任务要求和协同时序任务要求的复杂规划问题，

中国知网 https://www.cnki.net

提出一种由个体机器人在本地计算的执行策略求解方法，并能满足任务的时序与协同要求。该方法将计算量分摊到每个机器人，从而提高了任务规划的效率。进一步，针对机器人参与协同任务时因同步要求而产生的协同等待时间，本文提出一种基于分布式通信的执行策略迭代调整机制。机器人与其它机器人进行信息交互，并在本地迭代地调整自身的执行策略以优化在执行协同任务时的等待时间，进而有效提升多机器人的总时间效率。

（3）针对本文提出的多机器人时序任务规划框架，本文进行了相关的理论分析与仿真验证。本文分析了该规划框架的完备性，算法的计算复杂度，并讨论了影响执行策略最优性的因素。另外，本文在不同机器人数量、任务类型和环境规模下进行了大量的仿真实验，验证了本文中多机器人时序任务规划框架相较经典算法在可拓展性和求解效率上的显著优势。通过与基于混合整数规划模型的执行策略优化方法对比，验证了本文提出的执行策略迭代调整机制的快速收敛性与良好的策略优化效果。

6.2 下一步的研究方向

在现有的研究基础上，可以考虑从以下方面进一步拓展本文提出的规划方法的适用范围：

（1）动态环境中的机器人执行策略在线调整。本文考虑了静态环境中的复杂时序任务约束下的机器人执行策略求解与优化问题。然而在实际工作场景中，机器人面临的环境往往是动态变化的，比如移动的工作人员、动态变化的环境结构、在线出现的任务请求等。在这些任务场景中，机器人需要实时探测环境变化，并对自身的执行策略进行在线调整以满足任务要求。另外，机器人在线调整执行策略时还需要考虑任务之间的时序约束。如何在实际部署中优化执行任务的时间效率以及满足任务之间的时序约束，是需要进一步研究的重要科学问题。

（2）考虑机器人动力学约束的控制器设计。本文对工作环境进行了栅格化建模，并假设机器人在相邻栅格之间存在可行的转移路径，而未考虑机器人的实际动力学约束。在实际工作场景中，机器人的动力学约束将会影响其在不同区域之间的转移路径以及时间，从而影响任务规划的实际效果。另外，在多机器人协同场景中，需要进一步考虑机器人之间的安全距离等约束。如何将上层任务规划与底层运动控制器设计相结合，以保

中国知网 https://www.cnki.net

证机器人执行策略的可行性，是将多机器人实际部署到复杂任务场景中需要考虑的重要问题。

（3）考虑巡回任务的时间效率优化。本文考虑了在有限线性时序逻辑任务约束下，机器人的执行策略求解与优化问题。然而，完整的线性时序逻辑语法还可以描述如持续访问某些任务区域等巡回任务要求。在多机器人的任务协同中，如果出现某个机器人需要访问的巡回任务区域过多（访问周期过长），就会限制其它机器人的执行效率。在这种场景下，如何设计合适的优化指标，以及可能的任务均衡机制，来提升多机器人系统执行任务的效率，是拓展本文所提出的算法框架需要进一步研究的方向。

中国知网 https://www.cnki.net

参考文献

[1] Huang Y, Han H, Zhang B, et al. Supply distribution center planning in UAV-based logistics networks for post-disaster supply delivery[C]//Proceedings of the 2020 IEEE International Conference on E-health Networking, Application Services (HEALTHCOM). 2021: 1-6.

[2] Yu C, Lin B, Guo P, et al. Deployment and dimensioning of fog computing-based Internet of vehicle infrastructure for autonomous driving[J]. IEEE Internet of Things Journal, 2019, 6(1): 149-160.

[3] Chen X, Li Y, Liu L. A coordinated path planning algorithm for multi-robot in intelligent warehouse[C]//Proceedings of the 2019 IEEE International Conference on Robotics and Biomimetics (ROBIO). IEEE, 2019: 2945-2950.

[4] Bolu A, Korçak Ö. Adaptive task planning for multi-robot smart warehouse[J]. IEEE Access, 2021, 9: 27346-27358.

[5] Jiang Y, Mi Z, Wang H. An improved OLSR protocol based on task driven used for military UAV swarm network[C]//Proceedings of the 2019 International Conference on Intelligent Robotics and Applications(ICIRA). Springer International Publishing, 2019: 569-581.

[6] Silano G, Báca T, Penicka R, et al. Power line inspection tasks with multi-aerial robot systems via signal temporal logic specifications[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 4169-4176.

[7] Zhu D, Zhou B, Yang S X. A novel algorithm of multi-AUVs task assignment and path planning based on biologically inspired neural network map[J]. IEEE Transactions on Intelligent Vehicles, 2021, 6(2): 333-342.

[8] 杨杰, 席建祥, 王成, 等. 多无人机协同巡视任务规划方法综述[J]. 飞行力学, 2018(36): 1-6.

[9] 李大东, 孙秀霞, 孙彪, 等. 基于混合整数线性规划的无人机任务规划[J]. 飞行力学, 2010, (05): 90−93.

[10] Abichandani P, Benson H Y, Kam M. Mathematical programming approaches for multi-vehicle motion planning: linear, nonlinear, and mixed integer programming[J]. Foundations and Trends in Robotics, 2013, 2(4): 261-338.

[11] 梁国伟, 王社伟, 赵雪森. 多无人机协同任务分配方法[J]. 火力与指挥控制, 2014, 11.

[12] Asfora B A, Banfi J, Campbell M. Mixed-integer linear programming models for multi-robot non-adversarial search[J]. IEEE Robotics and Automation Letters, 2020, 5(4): 6805-6812.

[13] Yi J, Ahn M S, Chae H, et al. Task planning with mixed-integer programming for multiple cooking task using dual-arm robot[C]//Proceedings of the 2020 17th International Conference on Ubiquitous Robots (UR). 2020: 29-35.

中国知网 https://www.cnki.net

[14] Forsmo E J, Grøtli E I, Fossen T I, et al. Optimal search mission with unmanned aerial vehicles using mixed integer linear programming[C]//Proceedings of the International Conference on Unmanned Aircraft Systems (ICUAS). IEEE, 2013: 253-259.

[15] Milot A, Chauveau E, Lacroix S, et al. Market-based multi-robot coordination with HTN planning[C]//Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS. IEEE, 2021: 2606-2612.

[16] Gautier P, Laurent J, Diguet J. Comparison of market-based and DQN methods for multi-robot processing task allocation (MRpTA)[C]//Proceedings of the Fourth IEEE International Conference on Robotic Computing, IRC. IEEE, 2020: 336-343.

[17] 邸斌, 周锐, 丁全心. 多无人机分布式协同异构任务分配[J]. 控制与决策, 2013, (28(02)): 274-278.

[18] Liang H, Kang F. A novel task optimal allocation approach based on contract net protocol for agent-oriented UUV swarm system modeling[J]. Optik, 2016, 127(8): 3928-3933.

[19] 王和平, 柳长安, 李为吉. 基于蚁群算法的无人机任务规划[J]. 西北工业大学学报, 2005, 23(1).

[20] Li W, Zhao T, Dian S. A multi-robot coverage path planning method based on genetic algorithm[C]//Proceedings of the 2021 International Conference on Security, Pattern Analysis, and Cybernetics, SPAC. IEEE, 2021: 13-18.

[21] 杜健健, 万晓冬. 基于蝙蝠算法的多无人机协同侦察任务规划[J]. 电子测量技术, 2019, 7: 40-43.

[22] Zaza T, Richards A. Ant colony optimization for routing and tasking problems for teams of UAVs[C]//Proceedings of the UKACC International Conference on Control (CONTROL). IEEE, 2014: 652-655.

[23] Martin J, Frejo J, García R. Multi-robot task allocation problem with multiple nonlinear criteria using branch and bound and genetic algorithms[J]. Intelligent Service Robotics, 2021, 14: 707-727.

[24] 肖云涛. 基于线性时序逻辑的最优巡回路径规划方法与应用[D]. 浙江工业大学, 2014.

[25] 焦梦甜. 线性时序逻辑约束下的智能体路径规划方法研究[D]. 河南理工大学, 2019.

[26] 范振雍. 满足时序任务约束的机器人路径规划方法优化与研究[D]. 浙江工业大学, 2020.

[27] Baier C, Katoen J P. Principles of model checking (Representation and mind series)[M]. The MIT Press, 2008.

[28] Kress-Gazit H, Lahijanian M, Raman V. Synthesis for robots: Guarantees and feedback for robot behavior[J]. Annual Review of Control, Robotics, and Autonomous Systems, 2018, 1(1): 211-236.

[29] Fainekos G E, Kress-Gazit H, Pappas G J. Temporal logic motion planning for mobile robots[C]//Proceedings of the 2005 IEEE International Conference on Robotics and Automation. 2005: 2020-2025.

中国知网 https://www.cnki.net

[30] Smith S L, Tůmová J, Belta C, et al. Optimal path planning for surveillance with temporal-logic constraints[J]. The International Journal of Robotics Research, 2011, 30(14): 1695-1708.

[31] Ulusoy A, Smith S L, Ding X C, et al. Optimality and robustness in multi-robot path planning with temporal logic constraints[J]. The International Journal of Robotics Research, 2013, 32(8): 889-911.

[32] Khalidi D, Gujarathi D, Saha I. T* : A heuristic search based path planning algorithm for temporal logic specifications[C]//Proceedings of the 2020 IEEE International Conference on Robotics and Automation. 2020: 8476-8482.

[33] Kantaros Y, Zavlanos M M. Intermittent connectivity control in mobile robot networks[C]//Proceedings of the 2015 49th Asilomar Conference on Signals, Systems and Computers. 2015: 1125-1129.

[34] Schillinger P, Bürger M, Dimarogonas D V. Decomposition of finite LTL specifications for efficient multi-agent planning[G]//Distributed Autonomous Robotic Systems. Springer, 2018: 253-267.

[35] Schillinger P, Bürger M, Dimarogonas D V. Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems[J]. The International Journal of Robotics Research, 2018, 37(7): 818-838.

[36] Faruq F, Parker D, Laccrda B, et al. Simultaneous task allocation and planning under uncertainty[C]//Proceedings of the 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2018: 3559-3564.

[37] Banks C, Wilson S, Coogan S, et al. Multi-agent task allocation using cross-entropy temporal logic optimization[C]//Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA). 2020: 7712-7718.

[38] Leahy K, Jones A, Vasile C I. Fast decomposition of temporal logic specifications for heterogeneous teams[J]. ArXiv e-prints, 2020: 2010.00030.

[39] Plaku E, Karaman S. Motion planning with temporal-logic specifications: progress and challenges[J]. AI Communications, 2015, 29: 151-162.

[40] Luo X, Kantaros Y, Zavlanos M M. An abstraction-free method for multi-robot temporal logic optimal control synthesis[J]. IEEE Transactions on Robotics, 2021: 1-21.

[41] LaValle S M, et al. Rapidly-exploring random trees: a new tool for path planning[J]. Ames, IA, USA, 1998.

[42] Varricchio V, Chaudhari P, Frazzoli E. Sampling-based algorithms for optimal motion planning using process algebra[C]//Proceedings of the 2014 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2014: 5326-5332.

[43] Vasile C I, Belta C. Sampling-based temporal logic path planning[C]//Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2013: 4817-4822.

中国知网 https://www.cnki.net

[44] Kantaros Y, Zavlanos M M. Sampling-based control synthesis for multi-robot systems under global temporal specifications[C]//Proceedings of the 2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS). 2017: 3-14.

[45] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning[J]. The International Journal of Robotics Research, 2011, 30(7): 846-894.

[46] Kantaros Y, Zavlanos M M. Sampling-based optimal control synthesis for multirobot systems under global temporal tasks[J]. IEEE Transactions on Automatic Control, 2019, 64(5): 1916-1931.

[47] Kantaros Y, Zavlanos M M. STyLuS*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems[J]. The International Journal of Robotics Research, 2020, 39(7): 812-836.

[48] Kantaros Y, Zavlanos M M. Distributed optimal control synthesis for multi-robot systems under global temporal tasks[C]//Proceedings of the 2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS). 2018: 162-173.

[49] Schillinger P, Bürger M, Dimarogonas D V. Hierarchical LTL-task MDPs for multi-agent coordination through auctioning and learning[J]. The International Journal of Robotics Research, 2019.

[50] Bozkurt A K, Wang Y, Zavlanos M M, et al. Model-free reinforcement learning for stochastic games with linear temporal logic objectives[C]//Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA). 2021: 10649-10655.

[51] Cai M, Xiao S, Li B, et al. Reinforcement learning based temporal logic control with maximum probabilistic satisfaction[C]//Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA). 2021: 806-812.

[52] Sahin Y E, Nilsson P, Ozay N. Provably-correct coordination of large collections of agents with counting temporal logic constraints[C]//Proceedings of the 2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS). 2017: 249-258.

[53] Sahin Y E, Nilsson P, Ozay N. Multirobot coordination with counting temporal logics[J]. IEEE Transactions on Robotics, 2020, 36(4): 1189-1206.

[54] Tumova J, Dimarogonas D V. Multi-agent planning under local LTL specifications and event-based synchronization[J]. Automatica, 2016, 70: 239-248.

[55] Moarref S, Kress-Gazit H. Decentralized control of robotic swarms from high-level temporal logic specifications[C]//Proceedings of the 2017 International Symposium on Multi-Robot and Multi-Agent Systems (MRS). 2017: 17-23.

[56] Guo M, Dimarogonas D V. Bottom-up motion and task coordination for loosely-coupled multi-agent systems with dependent local tasks[C]//Proceedings of the 2015 IEEE International Conference on Automation Science and Engineering (CASE). 2015: 348-355.

中国知网 https://www.cnki.net

[57] Guo M, Dimarogonas D V. Task and motion coordination for heterogeneous multiagent systems with loosely coupled local tasks[J]. IEEE Transactions on Automation Science and Engineering, 2017, 14(2): 797-808.

[58] Tumova J, Dimarogonas D V. Decomposition of multi-agent planning under distributed motion and task LTL specifications[C]//Proceedings of the 2015 54th IEEE Conference on Decision and Control (CDC). 2015: 7448-7453.

[59] Kantaros Y, Guo M, Zavlanos M M. Temporal logic task planning and intermittent connectivity control of mobile robot networks[J]. IEEE Transactions on Automatic Control, 2019, 64(10): 4105-4120.

[60] De Giacomo G, De Masellis R, Montali M. Reasoning on LTL on finite traces: insensitivity to infiniteness[C]//Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence. AAAI Press, 2014: 1027-1033.

[61] Yu P, Dimarogonas D V. Distributed motion coordination for multi-robot systems under LTL specifications[J]. IEEE Transactions on Robotics, 2021: 1-16.

[62] Gastin P, Oddoux D. Fast LTL to Büchi automata translation[C]//Lecture Notes in Computer Science: Proceedings of the 13th International Conference on Computer Aided Verification CAV: vol. 2102. Springer, 2001: 53-65.

[63] Duret-Lutz A, Lewkowicz A, Fauchille A, et al. Spot 2.0 — a framework for LTL and ω-automata manipulation[C]//Lecture Notes in Computer Science: Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA): vol. 9938. Springer, 2016: 122-129.

[64] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1: 269-271.

[65] Gurobi Optimization, LLC. Gurobi optimizer reference manual[Z]. 2021.

[66] De Moura L, Bjørner N. Z3: An efficient SMT solver[C]//Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Springer-Verlag, 2008: 337-340.

中国知网 https://www.cnki.net

攻读硕士学位期间的主要成果

论文：

1. Ruofei Bai, Ronghao Zheng, Meiqin Liu and Senlin Zhang, Multi-Robot Task Planning under Individual and Collaborative Temporal Logic Specifications[C]//Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021:6382-6389.

2. Ruofei Bai, Ronghao Zheng, Yang Xu, Meiqin Liu, and Senlin Zhang, Hierarchical Multi-robot Strategies Synthesis and Optimization under Individual and Collaborative Temporal Logic Specifications[J]. Robotics and Autonomous Systems, 2022:104085.

专利：

1. 郑荣濠、白若飞、刘妹琴、张森林，一种机器人时序任务规划方法及装置、电子设备，国家发明专利，受理号：202110802366.0，2021 年 7 月 15 日。（法律状态：公开）

软件著作：

1. 浙江大学（开发者：郑荣濠、白若飞、刘妹琴、张森林），登记号：2021SR1092366，机器人时序任务规划系统 V1.0

参与项目：

1. 国家自然科学基金面上项目: 面向协同任务的多自主机器人分布式规划问题研究（项目编号：No. 61873235），2019/01——2022/12.

2. 浙江省自然科学基金重点项目: 面向海洋监测的异构多 AUV 系统协同控制研究（项目编号：LZ19F030002），2019/01——2022/12.

3. 航天科学技术基金项目: 面向无人机协同作战的分布式任务规划问题研究，2019/01——2019/12.

中国知网 https://www.cnki.net

答辩委员会表决结果

多机器人协同任务规划是当前的研究热点，选题具有重大的理论研究意义和实际应用价值。

论文作者在阅读国内外相关文献的基础上，完成的主要工作和创新研究成果如下：（1）提出了基于时序逻辑语言的协同任务建模方法，并基于自动机理论对任务要求进行独立性分解与分配，降低了任务规划问题的规模与求解复杂度；（2）提出了一种基于本地计算的执行策略求解方法，适用于机器人同时具有个体任务和协同任务的复杂规划问题，并具有良好的可拓展性；（3）设计了一种分布式执行策略迭代调整机制，降低了多机器人执行协同任务的等待时间，提高了机器人执行任务的时间效率。

论文结构合理，条理清晰，表达准确，研究成果表明作者具有扎实的专业理论基础和较强的科研能力。

答辩时叙述清楚，回答问题正确。经答辩委员会无记名投票表决，一致同意白若飞同学通过论文答辩，同意毕业，同意授予工学硕士学位。

答辩委员会主席签字：

签字日期：2022年3月8日

中国知网 https://www.cnki.net